

**Diseño e Implementación de un Agente en Espacio de
Usuario**
**para la Gestión Dinámica de Frecuencia (DVFS) en
Sistemas**
**Heterogéneos mediante Modelos Ligeros de Machine
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Resumen

El consumo energético limita el escalamiento de la computación de alto rendimiento (HPC) en nodos que combinan CPU y GPU. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo para controlarlo, pero los gobernadores nativos de Linux deciden a partir de la utilización del procesador, una señal que no distingue si el rendimiento está limitado por el cómputo o por la memoria.

Este trabajo diseñó, implementó y evaluó un agente en espacio de usuario que infiere el régimen de ejecución (*compute-bound* o *memory-bound*) a partir de telemetría de hardware (contadores PMU y RAPL en CPU, NVML en GPU) y ajusta la frecuencia para reducir el Producto Energía–Retardo (EDP). El desarrollo tuvo cuatro fases. En la primera se construyó un instrumento de medición y un conjunto de datos etiquetado con el modelo Roofline. En la segunda se entrenaron clasificadores ligeros, validados dejando fuera cada familia algorítmica, con exactitud balanceada de 0.728 (XGBoost, CPU) y 0.819 (*random forest*, GPU), y se derivó la política de frecuencia de cada régimen. En la tercera se implementó el agente, y en la cuarta se evaluó frente a una base con los gobernadores *performance* y *powersave* en aplicaciones compuestas y en dos aplicaciones externas, LAMMPS y CloverLeaf.

En CPU la política resultó no actuar, porque cerca del 84 % de la potencia se mantiene al reducir el reloj. En GPU, fijar el reloj en 1260 MHz durante las fases *memory-bound* mejoró el EDP de la GPU en 8.9 %. En las dos aplicaciones con fases de memoria largas, la compuesta con familias vistas en el entrenamiento y CloverLeaf, el agente redujo el EDP del nodo en 4.42 % y 3.75 % (IC95 por bloques de 2.3 a 5.4 % y de 3.2 a 4.6 %); la prueba de signos declarada dio $p = 0,0625$, el mínimo posible con cinco bloques. En LAMMPS y en las familias no vistas el resultado quedó cerca de la paridad.

Palabras Clave: DVFS; eficiencia energética; computación de alto rendimiento; modelo Roofline; telemetría microarquitectónica; aprendizaje automático supervisado.

Abstract

Energy consumption limits the scaling of high performance computing (HPC) on nodes that combine CPUs and GPUs. Dynamic Voltage and Frequency Scaling (DVFS) is the main mechanism to control it, but the native Linux governors decide from processor utilization, a signal that does not distinguish whether performance is limited by computation or by memory.

This work designed, implemented and evaluated a user-space agent that infers the execution regime (*compute-bound* or *memory-bound*) from hardware telemetry (PMU and RAPL counters on CPU, NVML on GPU) and adjusts frequency to reduce the Energy-Delay Product (EDP). The work had four phases. In the first, a measurement instrument and a dataset labeled with the Roofline model were built. In the second, lightweight classifiers were trained and validated by leaving each algorithmic family out, reaching a balanced accuracy of 0.728 (XGBoost, CPU) and 0.819 (*random forest*, GPU), and a frequency policy was derived for each regime. In the third, the agent was implemented, and in the fourth it was evaluated against a baseline with the *performance* and *powersave* governors on composite applications and on two external applications, LAMMPS and CloverLeaf.

On CPU the policy was not to act, because about 84% of the power remains when the clock is lowered. On GPU, fixing the clock at 1260 MHz during *memory-bound* phases improved GPU EDP by 8.9%. On the two applications with long memory phases, the composite application built from families seen in training and CloverLeaf, the agent reduced node EDP by 4.42% and 3.75% (block-level 95% CIs of 2.3 to 5.4% and 3.2 to 4.6%); the pre-declared sign test gave $p = 0,0625$, the minimum attainable with five blocks. LAMMPS and the unseen families stayed close to parity.

Keywords: DVFS; energy efficiency; high performance computing; Roofline model; micro-architectural telemetry; supervised machine learning.

Índice general

Agradecimientos	1
Resumen	2
Abstract	3
Introducción	8
Planteamiento y justificación del problema	9
Objetivos	10
1. Marco de referencia	12
1.1. Marco Conceptual	12
1.1.1. Potencia CMOS y fundamento de DVFS	12
1.1.2. Modelo Roofline y regímenes de ejecución	13
1.1.3. Telemetría microarquitectónica con PMU	14
1.1.4. Espacio de usuario e interfaces de observabilidad	15
1.1.5. DVFS, P-states y mecanismos de actuación	16
1.1.6. Modelos de clasificación ligera e hiperparámetros	17
1.1.7. Producto Energía–Retardo (EDP)	18
1.2. Estado del Arte	18
2. Metodología	21
2.1. Enfoque y diseño	21
2.1.1. Población, muestra y criterios de inclusión	21
2.1.2. Plataforma experimental	22
2.1.3. Instrumento de medición	22
2.1.4. Modelo Roofline y etiquetado	23
2.2. Fase 1: recolección y caracterización	24
2.2.1. Verificación previa y catálogo	24
2.2.2. Matriz experimental y control de sesgos	25
2.2.3. Calidad de los datos y granularidad	25
2.2.4. Reproducibilidad	26
2.3. Fase 2: modelos de clasificación y política de frecuencia	26
2.3.1. Formulación, variables y validación	26
2.3.2. Decisión selectiva y calibración	28
2.3.3. Comparación y selección de modelos	28
2.3.4. Invarianza de las entradas frente a la acción del agente	29

2.3.5.	Derivación de la política de frecuencia	29
2.4.	Fase 3: agente de control DVFS	29
2.4.1.	Arquitectura	30
2.4.2.	Detección de fase, decisión y parámetros	31
2.4.3.	Política, actuación y coordinación	31
2.4.4.	Aplicaciones compuestas y verificación previa	32
2.5.	Fase 4: validación experimental	33
2.5.1.	Diseño de la matriz	33
2.5.2.	Escenarios adicionales	34
2.5.3.	Medición y análisis	34
3.	Resultados	37
3.1.	Resultados de CPU	37
3.1.1.	Composición de fase por kernel	38
3.1.2.	Variación de la composición con la frecuencia	39
3.1.3.	Techos Roofline y <i>ridge</i>	39
3.1.4.	Clasificador de fase en CPU	40
3.1.5.	Invarianza del clasificador de CPU frente a la acción del agente . . .	47
3.1.6.	Política de frecuencia por clase en CPU	47
3.2.	Resultados de GPU	49
3.2.1.	Validación de control y etiquetado	49
3.2.2.	Conjunto de datos y clasificador de fase en GPU	49
3.2.3.	Candidato operativo de GPU: invarianza frente a la acción del agente	51
3.2.4.	Política de frecuencia por clase en GPU	53
3.3.	Resultados del agente de control	56
3.3.1.	Candado de reloj de GPU y premisa física de la política	56
3.3.2.	Latencia, restauración y detección del agente	56
3.4.	Evaluación del agente frente a la base	57
3.4.1.	Aplicaciones compuestas	57
3.4.2.	Clasificación de fase con el agente en el lazo	58
3.4.3.	Tiempo, energía, EDP y costo del agente frente a la base	59
3.4.4.	Potencial de la GPU	60
3.4.5.	Escenario E: GPU dominada por fases de memoria	61
3.4.6.	Escenario D: LAMMPS como aplicación de terceros	64
3.4.7.	CloverLeaf CUDA Fortran como aplicación continua	65
4.	Discusión	67
4.1.	Validez de la medición	67
4.2.	Generalización de los clasificadores	67
4.3.	Alcance de la ganancia energética	68
5.	Conclusiones	69
5.1.	Limitaciones	70
5.2.	Trabajo futuro	71
	Bibliografía	72

Índice de tablas

2.1. Plataforma experimental.	22
2.2. Rejilla fina de frecuencia de la campaña definitiva	25
2.3. Brazos del agente.	31
2.4. Parámetros del agente.	31
2.5. Acciones del agente	32
2.6. Aplicaciones compuestas con fronteras y clases conocidas por construcción .	33
2.7. Matriz experimental de la Fase 4.	33
2.8. Mapa de los escenarios de la Fase 4	34
2.9. Amenazas a la validez y medidas de control	36
3.1. Configuración final del clasificador CPU.	42
3.2. Métricas del clasificador CPU	43
3.3. Métricas del clasificador GPU	52
3.4. Síntesis de la política GPU	54
3.5. Composición del escenario E	62

Índice de figuras

1.1. Modelo Roofline conceptual	13
1.2. Contadores de núcleo y contadores de <i>uncore</i>	16
1.3. Jerarquía de solicitud y actuación de DVFS	17
2.1. Recorrido de una campaña a través de las dos capas del instrumento	23
2.2. Validación anidada por familia	28
2.3. Arquitectura del agente	30
3.1. Composición de fase por familia en CPU	38
3.2. <i>Ridge</i> Roofline según el nivel de frecuencia CPU	39
3.3. Comparación de modelos del clasificador CPU	41
3.4. Matrices de confusión del clasificador CPU	43
3.5. Resultado del clasificador CPU por familia	44
3.6. Calibración y compromiso de la abstención en CPU	45
3.7. Perfiles de celdas confusables entre clases opuestas	46
3.8. Ganancia del EDP por nivel y clase, en forma de <i>forest plot</i>	48
3.9. Potencia de paquete y DRAM frente a la frecuencia	48
3.10. Comparación de modelos candidatos GPU	50
3.11. Invarianza de los candidatos GPU frente al reloj	51
3.12. Matrices de confusión del <i>random forest</i> de GPU	53
3.13. Ganancia del EDP GPU por nivel y clase, en forma de <i>forest plot</i>	55
3.14. Ciclo de cada aplicación compuesta	58
3.15. EDP del nodo relativo a la base en la matriz inicial	60
3.16. Potencial de la GPU en fases <i>memory_bound</i>	61
3.17. Escenario E: EDP del nodo y energía de GPU, con el contraste de gobernador en E-B	63
3.18. EDP mediano de LAMMPS por entrada y brazo	65
3.19. Resultados de CloverLeaf frente a la base	66

Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalamiento sostenible, especialmente en nodos heterogéneos que combinan CPU y GPU. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control en el hardware: permite ajustar el estado operativo de cada procesador para equilibrar la energía consumida y el tiempo de ejecución. Su efectividad depende, sin embargo, de que la frecuencia elegida corresponda al comportamiento de la aplicación, que cambia a lo largo de la ejecución.

Este problema se ha abordado con los gobernadores de frecuencia del sistema operativo y con heurísticas basadas en métricas microarquitectónicas. Los gobernadores de Linux deciden a partir de la utilización del procesador o de una política fija, sin considerar qué limita el rendimiento de la carga. Las heurísticas usan reglas de bajo costo, pero dependen de umbrales estáticos que generalizan mal cuando las fases de la aplicación cambian con rapidez. Ninguno de los dos enfoques distingue de forma explícita si una fase está limitada por la capacidad de cómputo del procesador (*compute-bound*) o por la transferencia de datos desde memoria (*memory-bound*), y de esa distinción depende cuánto se alarga la ejecución al reducir la frecuencia. Esta brecha motiva inferir el régimen durante la ejecución y decidir la frecuencia a partir de él.

En este contexto, el presente trabajo presenta el diseño, la implementación y la evaluación de un agente en espacio de usuario basado en modelos ligeros de aprendizaje automático. El agente clasifica la fase de ejecución como *compute-bound* o *memory-bound* a partir de telemetría microarquitectónica y ajusta la frecuencia de CPU y GPU para reducir el Producto Energía–Retardo (EDP). La evaluación lo compara con los gobernadores nativos de Linux que expone la plataforma y cuantifica el ahorro de energía, la degradación del rendimiento y el costo de observar e inferir.

Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [1] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [2].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases *compute-bound* y fases *memory-bound*. Esta distinción es crítica porque, en el régimen *memory-bound*, los incrementos de frecuencia producen mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético. En fases *compute-bound*, en cambio, la frecuencia sí impacta el desempeño de forma más directa. Por eso se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, y así habilitar decisiones de control más informadas [3, 4].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [5], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. Un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente es, por eso, una alternativa con valor técnico y práctico.

A nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. Por eso el problema es también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivos

Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS) en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.

3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del modelo en comparación con los gobernadores nativos de Linux.

Capítulo 1

Marco de referencia

1.1. Marco Conceptual

El marco conceptual reúne solo lo que el resto del trabajo utiliza. Parte de la relación entre frecuencia, voltaje, potencia y tiempo de ejecución, y del modelo Roofline, que distingue los regímenes *compute-bound* y *memory-bound* de una carga. Las señales de la PMU y de otros dominios del sistema alimentan un clasificador ligero que infiere ese régimen, y el Producto Energía–Retardo (EDP) permite valorar el efecto de las decisiones de frecuencia que se toman con esa inferencia.

1.1.1. Potencia CMOS y fundamento de DVFS

La potencia dinámica de un circuito CMOS depende del factor de actividad, la capacitancia efectiva, el voltaje de alimentación y la frecuencia de operación. Esta relación es la base física de DVFS y se expresa como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

donde α representa el factor de actividad, C la capacitancia efectiva conmutada, V el voltaje de alimentación y f la frecuencia de operación [6]. Para trabajar a frecuencias más altas normalmente se necesita un mayor voltaje. Por eso, cuando se reduce la frecuencia también es posible reducir el voltaje, y como el voltaje aparece al cuadrado en la ecuación, la potencia dinámica baja de forma marcada.

Sin embargo, la reducción de la potencia no asegura un menor consumo de energía,

porque la energía también depende del tiempo de ejecución. Si la aplicación tarda más en terminar a una frecuencia menor, parte del ahorro se pierde. Además, la potencia estática se consume durante toda la ejecución. Por eso el efecto energético de cambiar la frecuencia depende de cómo responda la carga, y esto motiva clasificar el régimen de cada carga.

1.1.2. Modelo Roofline y regímenes de ejecución

El modelo Roofline relaciona el rendimiento alcanzable de una carga con la capacidad de cómputo y el ancho de banda de memoria de la plataforma. La intensidad operacional decide cuál de los dos límites condiciona el rendimiento, según la expresión:

$$P = \min(P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

donde P_{pico} es el rendimiento aritmético pico, BW_{pico} el ancho de banda pico de memoria e I la intensidad operacional [7]. Si el rendimiento queda limitado por $I \cdot BW_{\text{pico}}$, la carga es *memory-bound*. Si el límite es P_{pico} , la carga es *compute-bound*. La Figura 1.1 muestra esta separación entre ambos regímenes.

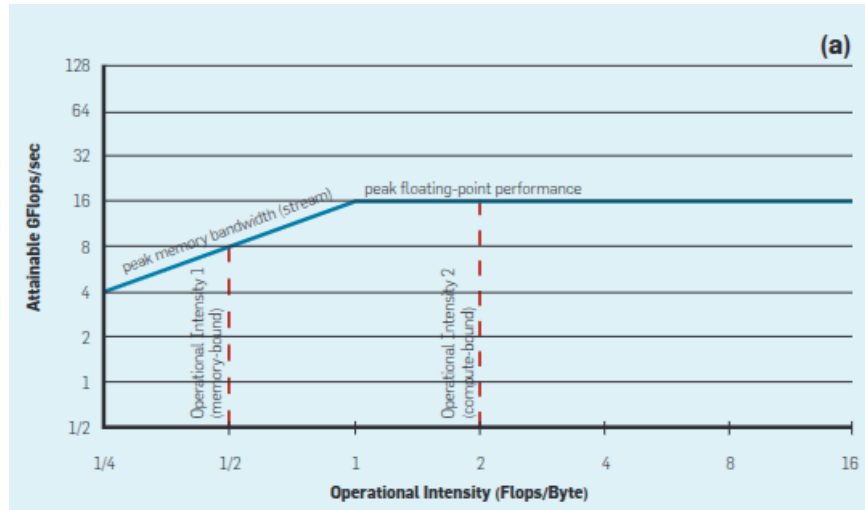


Figura 1.1: Modelo Roofline conceptual con el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Nota. Tomado de Williams et al. [7, Fig. 1(a)].

La intensidad operacional es la cantidad de operaciones por cada byte transferido desde el nivel de memoria considerado [7, p. 66]. No depende solo del tamaño del conjunto de datos ni de que este quepa en caché. La reutilización de datos reduce el tráfico hacia memoria y aumenta la intensidad operacional, incluso cuando el conjunto de trabajo supera la caché

[8, Secs. 3–4]. El modelo fija un límite de rendimiento para cada intensidad, pero una carga puede quedar por debajo de ese límite por otros efectos de la ejecución.

Los dos regímenes se separan en el punto de inflexión, o *ridge point*, donde el techo de cómputo y el límite de ancho de banda tienen el mismo valor. El umbral se obtiene como:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

Una intensidad menor que I_{ridge} corresponde al régimen *memory-bound* y una mayor, al *compute-bound* [7]. El *ridge point* es una frontera entre regímenes y no una frecuencia óptima de ejecución. Si cambia alguno de los techos, cambia también la posición de la frontera.

El techo de cómputo depende de la capacidad aritmética disponible, de la precisión de las operaciones y de la frecuencia de las unidades de cómputo [9]. El ancho de banda de memoria pertenece a otro dominio de la arquitectura y no cambia necesariamente en la misma proporción cuando se modifica la frecuencia [10, 5]. Por eso el I_{ridge} usado para clasificar debe corresponder a la precisión y al punto de operación de la observación. Los valores nominales sirven de referencia, pero la clasificación requiere techos medidos en las condiciones de ejecución [11].

La intensidad operacional observada se calcula con el trabajo aritmético y el tráfico hacia memoria medidos durante el mismo intervalo. Para un intervalo i es:

$$I_i = \frac{\text{FLOPs}_i}{B_i^{\text{DRAM}}} \quad (1.4)$$

donde FLOPs_i son las operaciones realizadas y B_i^{DRAM} los bytes transferidos desde o hacia la memoria principal en ese intervalo [7]. Los eventos que dan los FLOPs dependen de la microarquitectura y se interpretan según la semántica de la PMU [12]. El tráfico de DRAM se obtiene de contadores del controlador de memoria, cuyo ámbito puede diferir del de los contadores de núcleo. Ambas mediciones deben corresponder al mismo intervalo para que la intensidad operacional conserve su significado físico.

1.1.3. Telemetría microarquitectónica con PMU

La clasificación en CPU requiere señales que describan el comportamiento de la ejecución sin reconstruir en línea la intensidad operacional, cuya medición exige condiciones que un agente en operación no puede asumir (sección 2.3.1). Las Unidades de Monitorización

de Rendimiento (PMU) las proporcionan mediante contadores de hardware que acumulan eventos como instrucciones retiradas, ciclos y actividad de la jerarquía de memoria. Los contadores fijos registran eventos arquitectónicos predefinidos y los programables se configuran para eventos específicos de la microarquitectura [13, 14]. Una PMU no interpreta el comportamiento de la aplicación, porque entrega conteos cuyo valor depende del evento y del dominio observado. El mismo número solo adquiere significado al conocer qué recurso cuenta, durante qué intervalo y con qué alcance.

La multiplexación ocurre cuando se solicitan más eventos de los que el hardware puede medir al mismo tiempo. El sistema alterna grupos de eventos, cada contador queda activo solo durante una parte de la ventana y el valor registrado se ajusta según el tiempo que estuvo activo [15, 16]. Esa estimación pierde precisión si la aplicación cambia de fase mientras un evento no se mide. Por eso conviene limitar la cantidad de señales simultáneas, y esa limitación fija el presupuesto de eventos del instrumento.

Los contadores también difieren en su dominio. Los de núcleo observan recursos asociados a un núcleo y pueden vincularse con la ejecución de una tarea. Los de *uncore* cubren componentes compartidos del zócalo, como el controlador de memoria integrado y partes de la interconexión, y desde ellos se observa el movimiento de datos hacia memoria principal. Como su alcance es compartido, el tráfico que registran no se puede atribuir a un proceso individual sin condiciones de aislamiento. Además, ambas fuentes pueden tener granularidades temporales distintas y no deben combinarse fila a fila sin establecer antes una correspondencia temporal. La Figura 1.2 muestra esta diferencia estructural.

1.1.4. Espacio de usuario e interfaces de observabilidad

Un agente en espacio de usuario no accede directamente a los recursos protegidos del sistema, como los contadores de hardware o los registros de frecuencia [17, ch. 2]. Depende de las interfaces que el sistema operativo expone para realizar operaciones controladas [18]. Esta condición determina cómo el agente consulta contadores, lee energía y solicita cambios de frecuencia sin modificar estructuras internas del kernel.

En Linux, los eventos de la PMU se obtienen con `perf_event_open`, que permite configurar los contadores y consultar sus valores durante la ejecución [16]. Su disponibilidad depende del soporte del procesador y de los permisos del sistema.

Estos conteos describen la actividad del procesador, pero no la energía. Para medirla se usa Intel RAPL (*Running Average Power Limit*), que expone contadores de energía de varios dominios, entre ellos el paquete del procesador y, cuando está disponible, la memoria DRAM

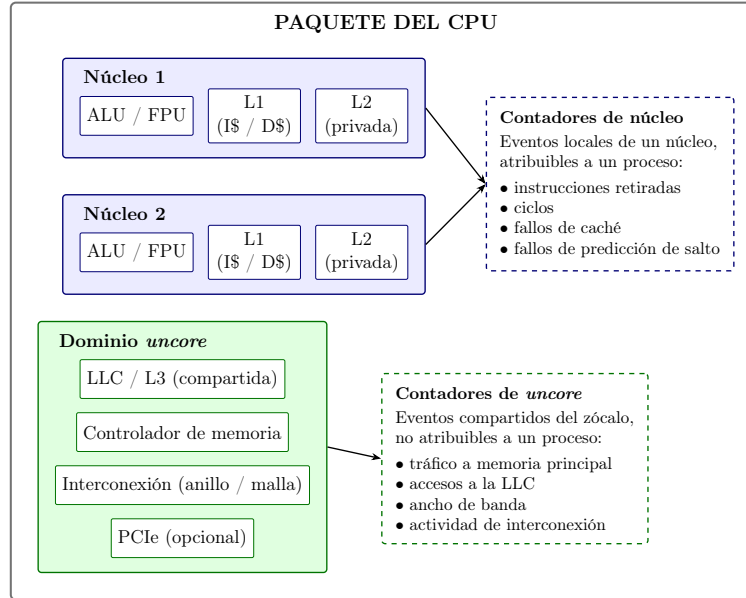


Figura 1.2: Ámbito de atribución de los contadores de núcleo y de *uncore* dentro del paquete del procesador. Fuente: Elaboración propia.

[19, 20]. Sus valores son energía acumulada, y la diferencia entre dos lecturas da el consumo de un intervalo, que luego se relaciona con métricas como el EDP.

1.1.5. DVFS, P-states y mecanismos de actuación

El escalado dinámico de voltaje y frecuencia (*Dynamic Voltage and Frequency Scaling*, DVFS) modifica el punto operativo del procesador para adaptar su capacidad de cómputo y su consumo a la carga. Los mecanismos de gestión de potencia reciben una solicitud de rendimiento y la materializan sobre el hardware [21].

En CPU, los *Performance States* o P-states son niveles de rendimiento con el procesador activo. P0 es el mayor y los siguientes reducen la capacidad disponible. Sobre ellos operan los *governors*, políticas de software que deciden qué nivel solicitar según el estado del sistema. En Linux, **performance** y **powersave** favorecen extremos opuestos del rango, **ondemand**, **conservative** y **schedutil** ajustan la solicitud de forma dinámica, y **userspace** permite que una aplicación pida el nivel deseado [21, 22].

La Figura 1.3 muestra el recorrido de una solicitud desde la interfaz de espacio de usuario, pasando por las políticas y los controladores del sistema, hasta los mecanismos que actúan sobre el hardware. Por eso una decisión de rendimiento no se traduce necesariamente de forma inmediata en la frecuencia observada, y el agente debe verificarla después de cada actuación.

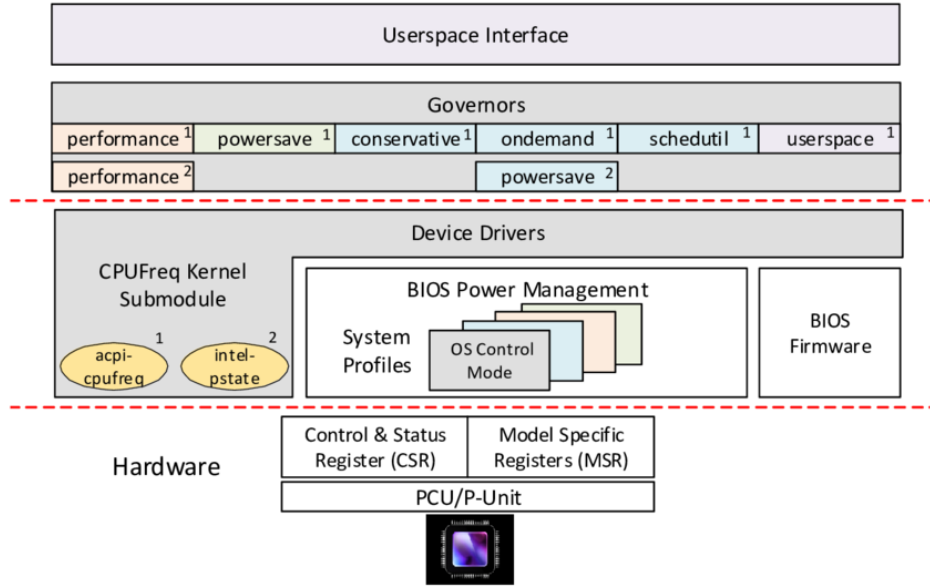


Figura 1.3: Jerarquía mediante la cual una política de rendimiento solicita cambios que posteriormente son materializados por el hardware. Fuente: Tomado de Hebbar y Milenković [21].

En GPU, DVFS mantiene el mismo principio, pero la actuación suele hacerse sobre dominios de reloj distintos, principalmente el de cómputo y el de memoria. Los niveles de frecuencia disponibles y la forma de aplicar una solicitud dependen de los mecanismos de gestión de la arquitectura y del controlador [10, 5]. Como en CPU, la transición entre niveles toma tiempo, así que el cambio de frecuencia no es instantáneo [23].

1.1.6. Modelos de clasificación ligera e hiperparámetros

Las observaciones obtenidas al ejecutar cargas acotadas, como *kernels* [7], *benchmarks* y *microbenchmarks* [24], se representan mediante un conjunto de características y una clase asociada. Esa relación se aprende con clasificación supervisada para inferir la clase de nuevas observaciones. Cuando la inferencia debe tener un costo reducido conviene usar modelos ligeros, cuya complejidad mantiene un tiempo de inferencia bajo sobre pocas características.

La regresión logística establece una frontera lineal de bajo costo y sirve como referencia [25, Ch. 4]. Un árbol de decisión introduce relaciones no lineales mediante divisiones sucesivas y es fácil de interpretar, aunque su complejidad crece con la profundidad [25, Ch. 8]. Random Forest combina varios árboles generados por muestreo y selección aleatoria de características [26]. Extra-Trees agrega aleatoriedad en los puntos de corte [27]. XGBoost construye los árboles de forma secuencial, de modo que cada uno corrige errores de los anteriores [28]. Estas familias cubren desde una frontera lineal hasta ensambles de árboles, y en todas el

costo de inferencia depende de la configuración.

Los hiperparámetros, como el número de árboles, la profundidad o la tasa de aprendizaje, modifican la capacidad de ajuste y la complejidad del modelo. Optuna explora ese espacio de forma secuencial [29] y usa el estimador de Parzen estructurado (TPE) para proponer configuraciones con mayor probabilidad de mejorar el objetivo [30], sin recorrer todas las combinaciones.

1.1.7. Producto Energía–Retardo (EDP)

Los cambios de frecuencia mediante DVFS afectan tanto el consumo de energía como el tiempo de ejecución. Por eso reducir una de estas magnitudes no significa necesariamente que el resultado sea mejor. El Producto Energía–Retardo (*Energy–Delay Product*, EDP) relaciona ambas en una sola expresión:

$$EDP = E \cdot T$$

donde E representa la energía consumida y T el tiempo de ejecución [31]. Un EDP menor indica una relación más favorable entre energía y tiempo. Como dos configuraciones pueden tener un EDP parecido con energía y tiempo muy distintos, ambas magnitudes se revisan también por separado.

1.2. Estado del Arte

La gestión energética en sistemas de alto rendimiento ha pasado de enfoques estáticos de reducción de potencia a estrategias adaptativas que buscan preservar el rendimiento sin costos energéticos desproporcionados. El escalado dinámico de voltaje y frecuencia (DVFS) es uno de los mecanismos más estudiados para modificar el punto operativo de procesadores y aceleradores. Su aplicación no da el mismo resultado en todas las cargas, porque una reducción de frecuencia puede disminuir la potencia instantánea, pero también prolongar el tiempo de ejecución y anular el beneficio energético. Por eso buena parte de la investigación pasó de aplicar DVFS de forma uniforme a reconocer el comportamiento de la carga y ajustar la frecuencia según sus características.

Los primeros trabajos que analizaron DVFS sobre regiones concretas de una aplicación mostraron que mantener una misma frecuencia durante toda la ejecución podía desaprovechar ahorros. Calore et al. [4] estudiaron kernels y una aplicación HPC completa sobre una CPU

Intel Haswell y una GPU NVIDIA K80, y observaron que el efecto energético variaba entre regiones y arquitecturas. Evaluaron cambios de frecuencia a nivel de función y mostraron que la conveniencia de cada configuración dependía de la región ejecutada y de la plataforma.

Esa dependencia de la carga llevó a buscar formas de anticipar su respuesta sin recorrer todas las frecuencias. Guerreiro et al. [10] estudiaron aplicaciones GPGPU con métricas recolectadas en una configuración inicial y las usaron para estimar la respuesta ante cambios en los dominios de cómputo y memoria. Así redujeron la exploración del espacio DVFS y mostraron que las métricas de rendimiento permiten agrupar aplicaciones por su sensibilidad a la frecuencia y orientar la selección hacia configuraciones favorables en energía.

Ali et al. [5] propusieron un método automatizado y portable para seleccionar frecuencias de GPU a partir de métricas tomadas en la configuración predeterminada del dispositivo. Con ellas construyeron modelos de potencia y de tiempo de ejecución para estimar el comportamiento en otros estados DVFS sin ejecutar la aplicación en cada uno. La selección consideró juntos el consumo de energía y la degradación del desempeño, y no contaba como mejora una reducción de potencia acompañada de un aumento importante del tiempo. En las plataformas evaluadas, la propuesta redujo la energía en promedio 29.6 % con una pérdida de rendimiento de 5.2 % en NVIDIA GA100, y 22.6 % con una pérdida de 4.7 % en NVIDIA GV100 [5].

Estos enfoques reducen el espacio de búsqueda, pero parten de una caracterización previa con la que se anticipa el comportamiento posterior de la aplicación. Cuando la carga cambia durante la ejecución hace falta reconocer el régimen que domina en cada momento. Antici et al. [3] abordaron este problema con MCBound, que clasifica cargas como *memory-bound* o *compute-bound* a partir de información de ejecuciones HPC para caracterizar trabajos nuevos. Con datos del supercomputador Fugaku, la separación alcanzó una puntuación F1-macro de al menos 0.89 con un costo reducido para el sistema [3]. Este resultado indica que la naturaleza computacional de una carga puede inferirse de forma automática y no depende solo de una clasificación manual basada en el conocimiento previo de la aplicación.

MCBound es cercano al problema de identificar el régimen, aunque no gobierna los estados DVFS de CPU o GPU. Aun así, separa dos partes que aparecen en los trabajos revisados, primero reconocer el comportamiento de la carga y luego decidir cómo usar esa información para modificar el punto de operación. La segunda parte se ha explorado con aproximaciones que adaptan la frecuencia durante la ejecución.

Simsek et al. [1] estudiaron esta actuación dinámica en simulaciones astrofísicas sobre GPU. Instrumentaron SPH-EXA para medir el comportamiento energético de distintas regiones y modificar la frecuencia del acelerador en puntos determinados de la ejecución. Frente

a una frecuencia uniforme, redujeron el consumo por GPU hasta 7.82 % con una degradación del rendimiento de aproximadamente 2.95 %. El trabajo muestra un beneficio potencial al cambiar la frecuencia durante la ejecución, pero la decisión queda ligada a regiones conocidas e instrumentadas dentro de la aplicación.

DRLCAP es un marco de *frequency capping* para GPU basado en aprendizaje por refuerzo profundo, y representa un paso hacia un control *online* completo. Monitoriza información a nivel de sistema, detecta cambios de fase y ajusta el límite de frecuencia sin intervención manual, con una reducción promedio de 22 % en la energía de GPU y menos de 3 % de degradación de rendimiento en arquitecturas NVIDIA [32]. El resultado muestra que el control de frecuencia puede automatizarse con señales observadas en línea, aunque con aprendizaje por refuerzo profundo y solo en GPU.

Gran parte de los enfoques recientes de predicción y control dinámico se concentra en GPU, y CPU y GPU difieren tanto en sus mecanismos de frecuencia como en las señales disponibles para observar la ejecución. Calore et al. [4] consideran ambas arquitecturas, pero Guerreiro et al. [10], Ali et al. [5], Simsek et al. [1] y Wang et al. [32] se centran en aceleradores gráficos. Queda espacio para estrategias que integren ambas plataformas y usen una caracterización de bajo costo como apoyo a las decisiones DVFS durante la ejecución, sin instrumentación específica de cada aplicación.

Capítulo 2

Metodología

2.1. Enfoque y diseño

La investigación es cuantitativa, experimental y aplicada. Es experimental porque modifica de forma controlada los estados de frecuencia de CPU y GPU para medir su efecto sobre el tiempo de ejecución, la energía y el Producto Energía–Retardo. Es aplicada porque, a partir de esas mediciones, se desarrolla y evalúa un agente de software para la gestión dinámica de frecuencia. El estado del dispositivo sin actuación del agente es el **escenario base**, identificado como REF en la rejilla de frecuencias, las tablas por nivel y las figuras. En la Fase 4, esta referencia se mide con los dos gobernadores nativos que expone `intel_pstate` en la plataforma: *performance*, con una frecuencia fija de 3.2 GHz en los núcleos delegados, y *powersave*, con el perfil de energía-rendimiento (EPP) en *default* y un rango de 0.8 a 3.2 GHz. En ambos casos el turbo de CPU permanece desactivado y la GPU conserva la gestión automática del controlador. El turbo se desactiva para evitar ajustes autónomos que impidan verificar la frecuencia efectiva durante la medición. El trabajo se organiza en cuatro fases encadenadas. La primera construye el conjunto de datos etiquetado. La segunda entrena y valida los clasificadores, los exporta a ONNX y deriva las tablas de política. La tercera implementa el agente y la cuarta lo evalúa experimentalmente. El material complementario de este capítulo está en el repositorio del proyecto (sección 2.2.4).

2.1.1. Población, muestra y criterios de inclusión

Se estudió una muestra intencional de cargas científicas reproducibles para CPU y GPU, elegidas para representar regímenes de ejecución contrastantes. Cada carga debía verificar su resultado, contener trabajo de punto flotante medible por el instrumento y permitir una

intensidad operacional válida para el etiquetado. La diversidad y la validación se definieron por familia algorítmica, y las variantes de tamaño de un mismo algoritmo no se contaron como familias independientes. El catálogo y su proceso de selección se presentan en la sección 2.2.1.

2.1.2. Plataforma experimental

Todos los experimentos se ejecutaron en un único nodo con asignación exclusiva mediante Slurm, para que cargas ajenas no compartieran los recursos durante la medición. La plataforma (Tabla 2.1) ofrece las interfaces para observar energía y telemetría en CPU y GPU mediante RAPL y NVML. Las actuaciones de frecuencia de CPU se aplicaron a los núcleos delegados y a sus hilos hermanos, porque ambos comparten el dominio físico de frecuencia. La sección 3.1 presenta la verificación de esta condición.

Tabla 2.1: Plataforma experimental.

Componente	Descripción
CPU	$2 \times$ Intel Xeon Gold 5315Y (Ice Lake-SP), 8 núcleos físicos por zócalo, 2 hilos por núcleo, 2 nodos NUMA
Frecuencia	<code>intel_pstate</code> , 0.8 a 3.2 GHz (3.6 GHz con <i>turbo</i> activo), dominio por núcleo físico
Energía de CPU	Intel RAPL, dominios de paquete y de DRAM por zócalo
GPU	NVIDIA A100 PCIe 40 GB, telemetría por NVML
Ejecución	Nodo en asignación exclusiva (Slurm)

2.1.3. Instrumento de medición

El instrumento propio tiene dos capas (Figura 2.1). La capa C++ adquiere los datos. Inicia la carga como un proceso detenido, configura los contadores de hardware sobre su árbol de procesos e hilos y la reanuda cuando la medición está preparada. Un hilo productor realiza las lecturas periódicas y un hilo consumidor las persiste, y ambos se comunican mediante una estructura circular de un productor y un consumidor. La capa Python coordina el protocolo experimental. Interpreta el manifiesto de campaña, verifica la plataforma, realiza la calibración, planifica las corridas, aplica los estados de frecuencia, valida cada ejecución y construye el conjunto de datos.

Una **ventana de muestreo** es el intervalo entre dos lecturas consecutivas y contiene la diferencia entre los valores acumulados en ambos extremos. En CPU se usaron hasta nueve eventos simultáneos. Ese número sale del presupuesto de contadores físicos del nodo y se veri-

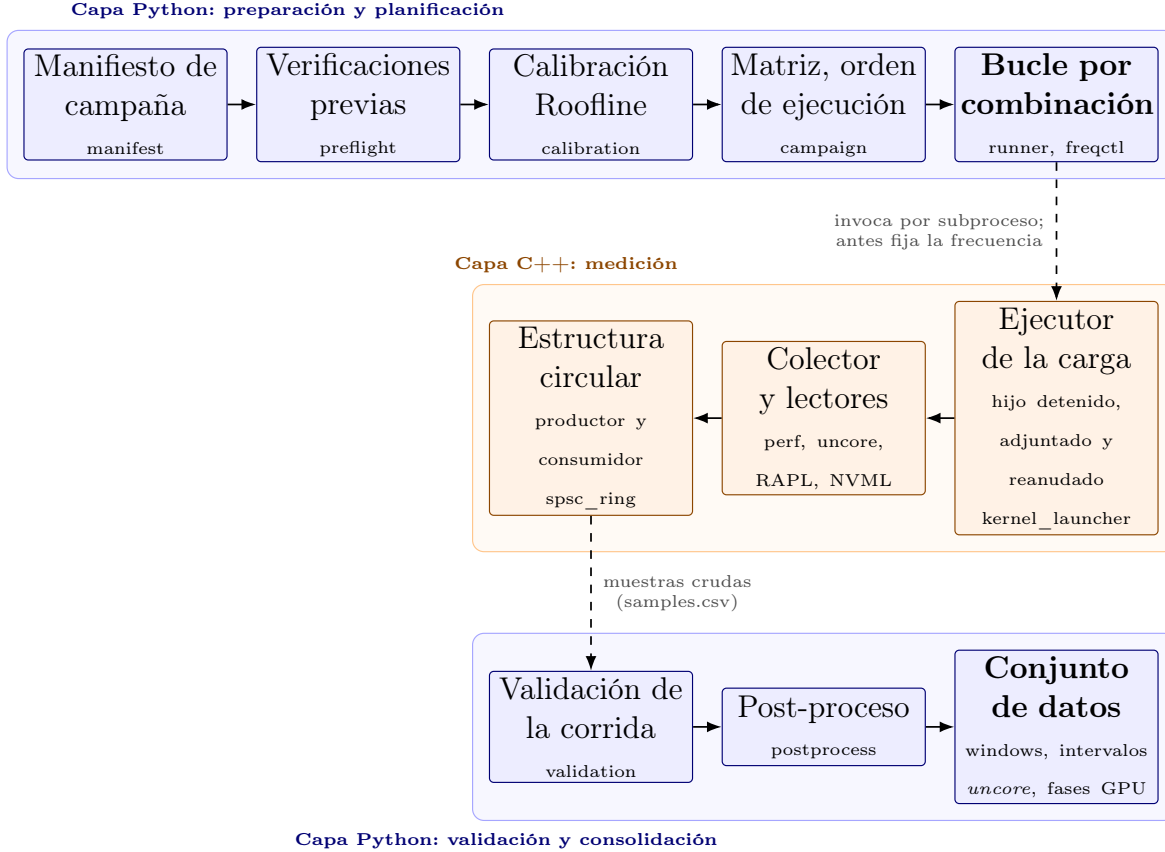


Figura 2.1: Recorrido de una campaña a través de las capas de preparación, medición y consolidación del instrumento.

ficó experimentalmente para evitar la multiplexación y la extrapolación de las mediciones. El tráfico a DRAM se obtiene de los contadores *uncore* del controlador de memoria, con intervalos de al menos 10 ms por el costo de adquisición a escalas menores [33]. Como el dominio *uncore* es compartido por todo el zócalo [34], cada campaña requiere asignación exclusiva del nodo para atribuir el tráfico a la carga medida. En GPU, el recolector solicita lecturas NVML cada 5 ms, pero los sensores se actualizan aproximadamente cada 100–120 ms. Por eso estas muestras se usan como telemetría agregada y no como ventanas GPU con resolución de 5 ms. Además, se registran la frecuencia observada de cada núcleo (`scaling_cur_freq`), la energía RAPL de paquete y de DRAM y, en GPU, la utilización de cómputo y de memoria, la potencia, el reloj del multiprocesador, la temperatura y la energía acumulada por NVML.

2.1.4. Modelo Roofline y etiquetado

La etiqueta de cada observación (*compute-bound* o *memory-bound*) resulta de comparar su intensidad operacional I con el punto de inflexión del modelo Roofline (ecuación (1.3)).

Los techos se calibran al inicio de cada campaña, con la misma asignación de recursos de las corridas y según la precisión aritmética. En los barridos de frecuencia, la calibración se ajusta además al nivel correspondiente. Para ello se usan microbenchmarks de ancho de banda y de cómputo, contrastados con una referencia declarada. La expectativa cualitativa de cada carga se conserva solo como referencia y no define la etiqueta.

La obtención de I difiere entre CPU y GPU. En CPU, los FLOPs de cada ventana suman los subeventos escalares y vectoriales de 128, 256 y 512 bits de `FP_ARITH_INST_RETIRED` [35], ponderados por el número de operandos de doble precisión de cada instrucción (ecuación (2.1)). Los bytes salen de los comandos CAS del controlador de memoria, así que I se calcula por intervalo durante la ejecución. En GPU, NVML no entrega FLOPs ni bytes por ventana o lanzamiento. Por eso la intensidad operacional se caracteriza fuera de línea con Nsight Compute [36], y el resultado se asocia a cada corrida. Para un kernel, esa intensidad suma sobre sus L lanzamientos perfilados las instrucciones de suma, multiplicación y FMA (esta última con dos FLOPs) y las divide por los bytes B_ℓ movidos a la memoria del dispositivo (ecuación (2.2)). Se considera estable para una configuración fija después de verificar su convergencia. Además, se distingue la precisión aritmética realmente ejercitada y se excluyen las cargas de aritmética entera o de precisión mixta, que no permiten una intensidad operacional consistente.

$$\text{FLOPs}_i^{\text{HW}} = \Delta N_{\text{esc},i} + 2\Delta N_{128,i} + 4\Delta N_{256,i} + 8\Delta N_{512,i} \quad (2.1)$$

$$I^{\text{GPU}} = \frac{\sum_{\ell=1}^L (N_\ell^{\text{add}} + N_\ell^{\text{mul}} + 2N_\ell^{\text{fma}})}{\sum_{\ell=1}^L B_\ell} \quad (2.2)$$

2.2. Fase 1: recolección y caracterización

2.2.1. Verificación previa y catálogo

Antes de cada campaña se ejecuta una batería de verificaciones automáticas sobre el entorno, el catálogo, la instrumentación y los permisos de actuación, y las verificaciones bloqueantes detienen la ejecución si falla una condición necesaria. El catálogo definitivo parte de familias elegibles sometidas a un cribado con tres niveles de frecuencia. El cribado descarta cargas no verificables, comprueba la calidad de las señales y cubre regímenes de ejecución contrastantes. El catálogo reúne cargas de suites públicas de HPC y cargas de elaboración propia, entre ellas familias con versión CPU y GPU del mismo algoritmo, y microbenchmarks de calibración que no forman parte del entrenamiento. La diversidad se controla por familia

algorítmica, con un mínimo de 5 familias por clase y dispositivo. Las familias, suites de origen, referencias, variantes y sumas de verificación están en el catálogo versionado del repositorio y en el material complementario (sección 2.2.4).

2.2.2. Matriz experimental y control de sesgos

La matriz experimental combina las cargas del catálogo, los estados de frecuencia de la Tabla 2.2 y tres repeticiones independientes por combinación. Los niveles se verifican contra los relojes soportados por el nodo, y en CPU el turbo permanece desactivado. Cada ventana conserva un estado de validez que distingue las observaciones aceptadas de las afectadas por la transición o por un nivel no aplicable.

Tabla 2.2: Rejilla fina de frecuencia de la campaña definitiva, con pasos uniformes de 300 MHz en CPU y de 150 MHz en GPU.

Dispositivo	Niveles de reloj en MHz
CPU	REF, F0 (3200), F1 (2900), F2 (2600), F3 (2300), F4 (2000), F5 (1700), F6 (1400), F7 (1100) y F8 (800).
GPU	REF, F0 (1410), F1 (1260), F2 (1110), F3 (960), F4 (810), F5 (660), F6 (510), F7 (360) y F8 (210).

La caracterización previa del cambio de frecuencia mostró tiempos de asentamiento del reloj de 10–11 ms en CPU y 70–80 ms en GPU. Con base en ellos, cada nivel se verifica dentro de una tolerancia de $\pm 5\%$ y se aplica un margen de guarda de 0.15 s al inicio y al final de la región medible.

Para reducir sesgos, el orden de ejecución se aleatoriza con una semilla registrada en el manifiesto [37]. Se incluyen repeticiones de una carga de referencia para detectar variaciones del entorno, y durante el barrido de un dispositivo el otro conserva su configuración de referencia.

2.2.3. Calidad de los datos y granularidad

Ninguna observación se elimina sin conservar su estado de calidad. Las muestras afectadas por calentamiento, contadores incompletos, energía inválida u otras condiciones de adquisición se identifican en el postproceso, y así el filtrado queda separado de la adquisición. El transitorio inicial de cada carga se estima sobre el IPC en CPU y sobre la utilización en GPU, y se excluye solo durante el análisis. Se toma el primer instante en que el coeficiente

de variación no supera 5 % en dos ventanas móviles consecutivas y se añade un margen de seguridad de 20 %. En GPU se exige además que la media de esas ventanas alcance 5 % de utilización. Si no aparece un estado estable, se aplica una detección recursiva de puntos de cambio y se elige el primer segmento con media de al menos 80 % de la meseta de mayor carga. Cada corrida también se valida por integridad de adquisición y por el criterio de éxito de la carga.

La unidad de entrenamiento depende del dispositivo. En CPU es un **intervalo del controlador de memoria**, sobre el que las variables se recalculan a partir de los deltas agregados. En GPU es una **corrida completa**, representada por agregados de las muestras válidas posteriores al calentamiento. Las lecturas de una misma corrida no se consideran observaciones independientes.

2.2.4. Reproducibilidad

Cada campaña se define mediante un manifiesto versionado que registra las cargas, los estados de frecuencia, las repeticiones, el período de muestreo, la asignación de núcleos, la semilla y las referencias de calibración. Junto con los datos se almacenan el perfil de capacidades del nodo, la calibración y los metadatos de cada corrida. Las ejecuciones interrumpidas se reanudan sin repetir corridas ya aceptadas, y una huella del protocolo impide combinar resultados obtenidos con configuraciones diferentes. El código, los datos procesados que respaldan cada tabla y figura, y el material complementario de este documento (catálogo completo, rejillas numéricas, protocolos por escenario y figuras adicionales) están en el repositorio público del proyecto, <https://github.com/AdrianCCRS/hyperion>, en el archivo `docs/libro/README.md`.

2.3. Fase 2: modelos de clasificación y política de frecuencia

2.3.1. Formulación, variables y validación

Se formula una clasificación binaria supervisada por dispositivo, cuya salida es la probabilidad de la clase *memory-bound*. En CPU se utilizan seis variables. Son el IPC, el MPKI, la tasa de fallos de caché, la fracción de ciclos detenidos por memoria, las instrucciones por segundo y la frecuencia observada. Esta última se conserva porque el cambio de frecuencia modifica el techo de cómputo y puede desplazar una carga respecto del punto de inflexión. Antes del entrenamiento se revisa la redundancia entre variables con la correlación y el factor

de inflación de la varianza (VIF). Se toman como criterios de alerta los pares con correlación absoluta superior a 0.85 y un VIF mayor a 10, y el conjunto final queda registrado en un contrato versionado por dispositivo. Cuando dos columnas describen la misma señal se conserva la medición físicamente más directa. La intensidad operacional, los FLOPs, los bytes, el punto de inflexión y la etiqueta no pueden ser entradas, y el entrenamiento se aborta si alguna aparece.

El agente no calcula la intensidad operacional durante la ejecución, aunque el instrumento de la Fase 1 la obtiene por intervalo, porque su medición depende de condiciones que un agente en operación no puede asumir. En CPU, los bytes de DRAM salen de los contadores *uncore* del controlador de memoria, cuyo ámbito es el zócalo completo [34]. Atribuirlos a una carga exige la asignación exclusiva del nodo, y leerlos requiere privilegios elevados (`CAP_PERFMON`). Los contadores de núcleo, en cambio, pueden asociarse a la carga que se ejecuta. En GPU, NVML no entrega FLOPs ni bytes, y la intensidad operacional solo se obtiene con Nsight Compute, que repite cada *kernel* perfilado para reunir sus métricas [36]. Durante la caracterización, perfilar el GEMM de RAJAPerf alargó su ejecución unas 60 veces. Por eso la intensidad operacional se usa solo para construir la etiqueta, en campañas con el nodo reservado o fuera de línea, y el clasificador la aproxima con señales que el agente puede leer a bajo costo durante la ejecución: contadores de núcleo en CPU y NVML en GPU.

La validación usa *leave-one-family-out* (LOFO). En cada pliegue se retira por completo una familia, que no interviene en el ajuste, en la selección de representación ni en la elección del umbral (Figura 2.2) [38, 39]. Para que las familias de mayor volumen no dominen el entrenamiento, cada celda de familia y clase se limita a 1000 intervalos con una semilla fija, y las observaciones se ponderan de forma inversa al tamaño de su celda, con media uno.

$$w_i = \frac{1/n_{fc}}{(1/N) \sum_{k=1}^N 1/n_{f_k c_k}} \quad (2.3)$$

La métrica principal es la exactitud media por celda de familia y clase [40],

$$\text{EB} = \frac{1}{|\mathcal{C}|} \sum_{(f,c) \in \mathcal{C}} \frac{a_{fc}}{n_{fc}}, \quad (2.4)$$

donde $\mathcal{C}$ es el conjunto de celdas observadas (cada combinación de familia f y clase c), $|\mathcal{C}|$ es el número de celdas, a_{fc} son los aciertos de cada celda y n_{fc} sus observaciones totales. Como complemento se reporta la puntuación F_1 macro agrupada. La incertidumbre se estima con cinco semillas de submuestreo y con *bootstrap* de familias con 2000 réplicas [41].

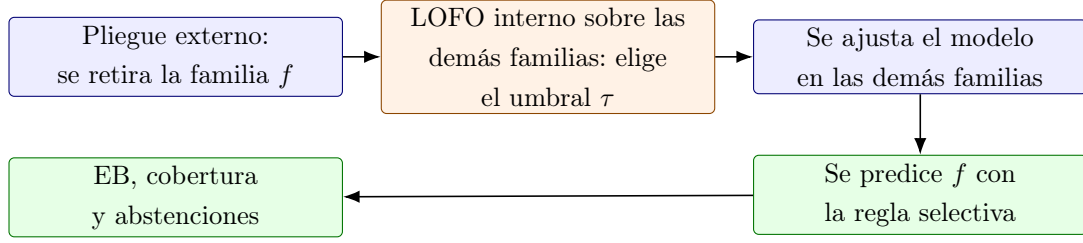


Figura 2.2: Validación anidada LOFO. La familia retirada no interviene en ninguna decisión de ajuste. Fuente: elaboración propia.

2.3.2. Decisión selectiva y calibración

Sea p_i la probabilidad de *memory-bound* y $q_i = \max(p_i, 1 - p_i)$ la confianza. El modelo emite una clase solo cuando $q_i \geq \tau$, y en caso contrario se abstiene con la salida *revisar*:

$$\hat{y}_i = \begin{cases} \text{memory-bound}, & p_i \geq 0,5 \wedge q_i \geq \tau, \\ \text{compute-bound}, & p_i < 0,5 \wedge q_i \geq \tau, \\ \text{revisar}, & q_i < \tau. \end{cases} \quad (2.5)$$

El umbral τ se selecciona con un LOFO interno sobre una rejilla de valores. Se maximiza la exactitud media por celda sobre las predicciones aceptadas y se exige una cobertura mínima de 0.60. La cobertura es la proporción de observaciones que reciben una clase y cuantifica el costo de la abstención. La calibración de la confianza se evalúa con el error de calibración esperado (ECE), calculado sobre diez intervalos de igual amplitud:

$$\text{ECE} = \sum_{b=1}^{10} \frac{n_b}{N} |\text{conf}_b - \text{acc}_b|, \quad (2.6)$$

donde N es el número total de observaciones evaluadas, n_b el número de observaciones del rango b , conf_b la confianza media de ese rango y acc_b su proporción de aciertos.

2.3.3. Comparación y selección de modelos

Bajo el mismo protocolo se comparan una regla mayoritaria, un árbol de profundidad uno, regresión logística, *random forest*, *extra trees* y XGBoost, y en GPU se añade un árbol de profundidad seis [42]. Los hiperparámetros principales se fijan antes del ajuste y, como análisis complementario, se evalúa una búsqueda anidada con Optuna dentro de cada pliegue externo. También se aplican diagnósticos de separabilidad y de capacidad del modelo, entre ellos un clasificador de vecinos más cercanos sin ajuste (sección 3.1.4). La selección final

considera el desempeño LOFO, la calibración, la cobertura y la estabilidad de las entradas frente a la acción del agente. Los modelos y umbrales resultantes se presentan en el capítulo de resultados.

2.3.4. Invarianza de las entradas frente a la acción del agente

Además del desempeño predictivo, se evalúa si la propia actuación del agente puede modificar indirectamente las entradas y generar retroalimentación. En GPU se aplican dos pruebas sobre los datos disponibles. La prueba contrafactual sustituye el valor de una entrada potencialmente afectada y mide los cambios de predicción. La prueba de exclusión repite la validación LOFO sin esa variable. Con ambas se decide si la entrada se conserva, se excluye o requiere una restricción operativa.

El modelo se evalúa de nuevo con el contrato de entradas resultante. En CPU se aplica la misma auditoría a la frecuencia observada, con sustituciones coherentes con el rango de actuación previsto. La latencia de inferencia se reporta por separado de la latencia completa entre detección y actuación. Los resultados de estas auditorías están en las secciones 3.2.3 y 3.1.5.

2.3.5. Derivación de la política de frecuencia

La política de frecuencia se deriva de forma independiente del clasificador, a partir de las mediciones de la Fase 1. Para cada clase, el EDP de cada nivel se compara con la base sobre el mismo kernel, y la ganancia agregada se estima mediante *bootstrap* de kernels con 4000 réplicas (intervalo de confianza de 95 %). Un nivel solo entra en la política si su mejora está respaldada estadísticamente y supera el efecto mínimo relevante de 1 %. Las diferencias pareadas por kernel se contrastan con la prueba de rangos con signo de Wilcoxon [43], o con la prueba *t* pareada cuando hay ocho o más pares y Shapiro–Wilk no rechaza la normalidad. La regla se valida dejando una familia fuera y se congela antes de evaluarse sobre familias externas. En GPU, la clase de un kernel es la mayoritaria entre sus corridas y se marca ambigua si su margen es menor que 0.75. Los kernels ambiguos cercanos al punto de inflexión se conservan en el análisis y los tamaños de un mismo algoritmo se agrupan por familia.

2.4. Fase 3: agente de control DVFS

La Fase 3 recibe congelados de la Fase 2 el modelo de cada dispositivo, su umbral de abstención y la tabla de política. El modelo y el umbral proceden de la validación del

clasificador, y la política, de las comparaciones de EDP entre niveles. Así el desempeño predictivo no se asocia directamente con la conveniencia de una acción DVFS. Los modelos se exportan a ONNX y se ejecutan desde C++ con ONNX Runtime, tras verificar que sus probabilidades y etiquetas coinciden con las de los modelos originales. Con estos artefactos, la Fase 3 implementa la detección, la decisión, la actuación, la coordinación y la restauración que luego se evalúan en la Fase 4.

2.4.1. Arquitectura

El agente se implementa con dos procesos C++ independientes, uno para CPU y otro para GPU, coordinados mediante una señal binaria (Figura 2.3). Así los bucles de control pueden tener escalas temporales distintas y la actuación de cada dispositivo queda aislada. Ambos procesos cargan al inicio los modelos, umbrales y tablas de política congelados.

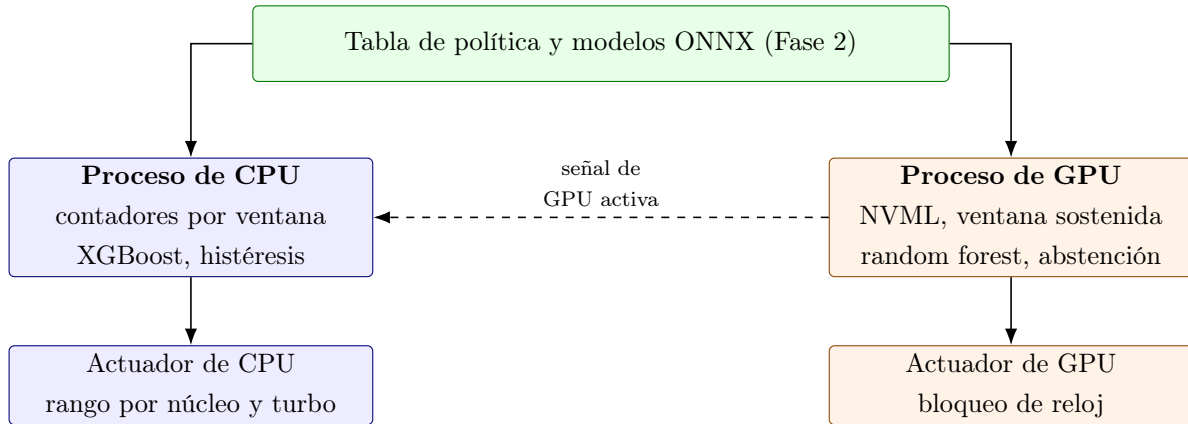


Figura 2.3: Arquitectura del agente: dos procesos, una tabla de política compartida y una señal de coordinación. Fuente: elaboración propia.

Cada proceso admite los tres brazos de la Tabla 2.3. El brazo *base* es la referencia contra la que se compara todo, y el brazo *sombra* mide la sobrecarga del agente sin su efecto sobre el hardware.

Tabla 2.3: Brazos del agente.

Brazo	Agente	Comportamiento
Base	No corre	En CPU, turbo desactivado y gobernador <i>performance</i> (3.2 GHz fijos) o <i>powersave</i> (EPP <i>default</i> , 0.8 a 3.2 GHz); en GPU, gestión automática del controlador
Sombra	Muestrea, clasifica y decide	Registra cada decisión y nunca escribe sobre el hardware
Activo	Muestrea, clasifica, decide y escribe	Aplica la frecuencia decidida y restaura el estado al terminar

2.4.2. Detección de fase, decisión y parámetros

El agente de GPU identifica episodios sostenidos de actividad con la utilización que reporta NVML, acumula 3 s de observación y decide una vez por episodio. Ante una abstención libera el reloj. En CPU la decisión se actualiza por ventana y una actuación requiere 50 propuestas consecutivas del mismo estado. La Tabla 2.4 resume los parámetros operativos.

Tabla 2.4: Parámetros del agente.

Proceso	Parámetro	Valor y criterio
GPU	Umbral de actividad	5 % de utilización
GPU	Ventana sostenida	3 s; una decisión por episodio de actividad
GPU	Cadencia de muestreo	50 ms
GPU	Umbral de abstención	0.90; seleccionado en la Fase 2 (sección 3.2.3)
GPU	Permanencia mínima	3.7 s; definida a partir de la latencia de actuación medida en la Fase 3 (sección 3.3.2)
CPU	Umbral de abstención	0.85; seleccionado en la Fase 2 (sección 3.1.4)
CPU	Histéresis	50 ventanas consecutivas
CPU	Piso con GPU activa	3.2 GHz; verificado en la Fase 3 (sección 3.3.2)

2.4.3. Política, actuación y coordinación

La Tabla 2.5 resume las acciones congeladas para la evaluación. La política operativa mantiene la actuación de GPU derivada en la Fase 2. En CPU se añaden dos variantes experimentales para estudiar el efecto de cerrar el lazo aunque la política derivada no recomiende reducir la frecuencia. Estas variantes no modifican el clasificador ni la política original y se usan solo como brazos de evaluación.

Tabla 2.5: Acciones del agente. Las filas de GPU provienen de la política derivada en la Fase 2; las filas de CPU son variantes experimentales evaluadas en la Fase 4.

Dispositivo	Clase	Acción
CPU	<i>compute_bound</i>	Fija F0 (3.2 GHz, turbo desactivado); experimental
CPU	<i>memory_bound</i>	Baja a F1 (2.9 GHz) con confianza mínima de 0.85; experimental
GPU	<i>memory_bound</i>	Fija el reloj en 1260 MHz (F1)
GPU	<i>compute_bound</i>	No actúa: libera el reloj al <i>governor</i> nativo
GPU	<i>revisar</i> (abstención)	No actúa: libera el reloj al <i>governor</i> nativo

El actuador de CPU reutiliza las restricciones de dominio de la Fase 1, aplica el cambio sobre los núcleos delegados y sus hilos SMT y verifica el estado después de cada actuación. Antes de modificar el dispositivo guarda su configuración inicial, y la restaura al finalizar o ante una terminación anómala. El actuador de GPU sigue la misma disciplina al fijar o liberar el reloj.

La coordinación entre ambos procesos es una regla de protección independiente de la política. Durante un episodio de actividad GPU, el proceso CPU mantiene un piso de 3.2 GHz sin esperar la histéresis, para que una reducción de frecuencia en una CPU que espera al acelerador no prolongue la ejecución GPU. Esta regla no modifica la clasificación ni la tabla de política, y su efecto sobre el EDP se evalúa por separado en la Fase 4.

2.4.4. Aplicaciones compuestas y verificación previa

Las aplicaciones compuestas registran, con reloj monotónico, fronteras de fase conocidas por construcción para contrastar las decisiones del agente (Tabla 2.6). La aplicación A usa familias presentes en el entrenamiento y la B, familias que no lo estuvieron. Una tercera aplicación alterna trabajo CPU y GPU. Cada fase mantiene al menos 3 s de actividad y se separa de la siguiente con 1 s de reposo para observar el cambio.

Tabla 2.6: Aplicaciones compuestas con fronteras y clases conocidas por construcción

Alcance	Aplicación	Fases (clase)
CPU	A (vistos)	dgemm_n2048 (<i>compute</i>), npb_cg (<i>memory</i>)
CPU	B (inéditos)	cpu_xsbench_omp (<i>memory</i>), cpu_rsbench_omp (<i>compute</i>)
GPU	A (vistos)	gpu_cutlass_simt_dgemm_n4096 (<i>compute</i>), gpu_rajaperf_stream_triad (<i>memory</i>)
GPU	B (inéditos)	BabelStream con -numtimes 1000 (<i>memory</i>), rodinia_lavamd con -boxes1d 100 (<i>compute</i>)

Antes de la evaluación final se definieron pruebas específicas para la latencia de actuación, la sobrecarga, la restauración ante terminación anómala y el cumplimiento de la regla de coordinación. Sus resultados están en la sección 3.3.

2.5. Fase 4: validación experimental

2.5.1. Diseño de la matriz

La matriz inicial evalúa el agente en tres dimensiones, que son el alcance experimental (CPU, GPU o ejecución conjunta), la procedencia de las familias (vistas durante el entrenamiento o inéditas) y el brazo de ejecución (Tabla 2.7). El orden de las celdas se aleatoriza con semilla fija y cada aplicación ejecuta dos ciclos completos por repetición. El brazo *sombra* estima la sobrecarga del agente sin actuación sobre el hardware. Los brazos *activo-F0* y *activo sin piso* son variantes experimentales que aíslan el efecto de la actuación y el de la regla de coordinación. Los alcances con tres repeticiones se analizan de forma descriptiva, con valores individuales y razones de medianas.

Tabla 2.7: Matriz experimental de la Fase 4.

Alcance	Brazos	Vistos	Inéditos	Rep.
CPU	base, sombra, activo, activo-F0	A	B	3
GPU	base, sombra, activo	A	B	3
Conjunto	base, sombra, activo con piso, activo sin piso	A	B	3 (5 en la matriz inicial)

2.5.2. Escenarios adicionales

Además de la matriz inicial se ejecutan los protocolos complementarios de la Tabla 2.8, cuyas condiciones completas están en el material complementario (sección 2.2.4). Cada protocolo se fija antes de sus mediciones y se reporta sin importar el resultado. Los escenarios D y CloverLeaf incorporan aplicaciones completas externas al catálogo inicial [44, 45]. El escenario E es dirigido, pues selecciona familias que en la Fase 2 mostraron una reducción de EDP de al menos 10 % en F1, y añade una variante con familias no vistas. Por eso sirve para estudiar el agente en condiciones favorables identificadas de antemano y no como estimación global de desempeño. En C y E se repite además la medición de la base como control de repetibilidad.

Tabla 2.8: Mapa de los escenarios de la Fase 4. El protocolo completo está en el repositorio.

Escenario	Contraste principal
Matriz inicial	Dispositivo, familias vistas e inéditas y brazo
C	Repetición independiente de CPU y conjunto
E	GPU con fases largas de memoria, familias vistas (E-A) e inéditas (E-B)
Repetición de base	Segunda medición de la base como control de repetibilidad
D	LAMMPS con cuatro entradas y fases naturales
F	Diagnóstico de la solicitud de F1 fijo
Confirmatorio E-A	Cinco bloques con base, sombra, agente y F1 fijo
E-B ampliado	Ocho bloques de E-B y diagnóstico de variabilidad de <i>myocyte</i> , con telemetría de CPU y GPU
E-B con <i>powersave</i>	Ocho bloques de E-B con los núcleos delegados en el gobernador <i>powersave</i> (EPP <i>default</i>), contraste de gobernador nativo
Serie <i>powersave</i>	Matriz inicial, E, confirmatorio E-A, D y CloverLeaf repetidos con los núcleos delegados en <i>powersave</i> , cada brazo frente a la base del mismo gobernador
CloverLeaf	Cinco bloques con base, sombra y agente en ejecución continua

2.5.3. Medición y análisis

En cada celda experimental se registran la duración, la energía de CPU mediante RAPL y la energía de GPU mediante NVML. El EDP del nodo se calcula como

$$\text{EDP} = (E_{\text{CPU}} + E_{\text{GPU}}) T,$$

y se conserva además $E_{\text{GPU}}T$ para los análisis específicos del acelerador. La ventana energética cubre toda la ejecución de la aplicación con los procesos del agente ya activos, y por eso incluye su sobrecarga. Al terminar cada celda se verifica que los procesos correspondientes hayan terminado y se restauran los estados de frecuencia modificados.

En la matriz inicial y en el escenario E, las aplicaciones compuestas registran fronteras de fase conocidas por construcción y las decisiones del agente, con instante, clase, confianza y actuación. LAMMPS se ejecuta con fases naturales sin etiquetar y CloverLeaf como aplicación continua. En ambos casos se conservan la telemetría y la actuación observada, pero no se asigna una verdad de fase independiente.

Para cada combinación experimental se calculan las medianas de duración, energía y EDP, y su razón respecto del brazo *base*. En la matriz conjunta, con observaciones no apareadas, el EDP se contrasta con una prueba bilateral de Mann–Whitney [43]. Los experimentos confirmatorios organizados por bloques se analizan con una prueba de signos exacta bilateral sobre el EDP [43]. Con cinco bloques, el menor valor p que puede dar esta prueba es $2 \cdot (1/2)^5 = 0,0625$, cuando todos los bloques favorecen al mismo brazo, así que solo informa la dirección del efecto. Para estimar su magnitud se calcula además la media geométrica de las razones brazo/base de los cinco bloques, con su intervalo de confianza de 95 % por una prueba t pareada sobre el logaritmo de esas razones [37]. Este análisis se añadió después de medir y no formaba parte del protocolo declarado, y con cinco bloques no es posible verificar el supuesto de normalidad del logaritmo. También se reportan las razones individuales por bloque en duración, energía y EDP para conservar la variabilidad entre repeticiones.

La clasificación se evalúa solo en las aplicaciones compuestas. Cada decisión se atribuye a la fase que contiene su instante, y se reportan aciertos, errores, abstenciones y cobertura por alcance, tipo de kernel y dispositivo. En CPU la puntuación es por ventanas, según la granularidad de decisión del agente. La comparación entre A y B muestra la variación del desempeño sobre familias no vistas dentro de este protocolo. En LAMMPS y CloverLeaf se reportan las decisiones y actuaciones observadas, pero no una exactitud de clasificación, porque no hay una referencia independiente de fase con esa granularidad.

La Tabla 2.9 resume las principales amenazas a la validez y las medidas adoptadas para controlarlas.

Tabla 2.9: Amenazas a la validez de mayor peso y medidas de control. La lista completa está en el material complementario del repositorio.

Amenaza	Medida de control
Interferencia de procesos ajenos en RAPL y en la GPU	Asignación exclusiva del nodo; ninguna carga adicional durante una medición
Frecuencia efectiva distinta de la pedida	Turbo apagado, rango por núcleo y hermanos <i>SMT</i> , verificación por ventana
Fuga de la etiqueta hacia las variables	Lista de columnas prohibidas comprobada por código
Pseudo-replicación entre tamaños y repeticiones	La familia como unidad de validación (LOFO) y de comparación de la política
Retroalimentación del agente sobre el modelo	Auditoría de invarianza de las variables frente a la acción del agente
Sesgo temporal o de orden	Orden aleatorizado con semilla registrada y repeticiones independientes
Muestra intencional en un solo nodo	Se declara como limitación; no se generaliza a otras plataformas

Capítulo 3

Resultados

Este capítulo presenta los resultados de las campañas de CPU y GPU, del agente de control y de su evaluación frente a la referencia en aplicaciones compuestas. Las tablas y figuras de sensibilidad, desglose y trazabilidad están en el material complementario del repositorio (sección 2.2.4); las referencias del tipo figura 7.1 del material complementario remiten a él. El cuerpo principal reúne la evidencia necesaria para sustentar cada conclusión.

3.1. Resultados de CPU

La campaña CPU cubrió 49 configuraciones de kernel en el nivel nativo y nueve niveles fijos de frecuencia, con tres repeticiones por combinación. Sus 1470 corridas quedaron aceptadas. Con los calentamientos calibrados, los transitorios fueron inferiores a 2 % de la duración de corrida en la mayoría de los kernels y de 4.8 % como máximo.

De 1 424 531 intervalos procesados, 1 309 755 (91.9 %) cumplieron simultáneamente los criterios de telemetría y de frecuencia definidos en Metodología. Se rechazaron 71 658 (5.0 %) por frecuencia no utilizable y 43 118 (3.0 %) por proceder de una ventana no válida. Los rechazados se conservaron para auditoría, pero no recibieron una etiqueta de entrenamiento. Entre los elegibles, 526 279 (40.2 %) son *compute_bound* y 783 476 (59.8 %) *memory_bound*.

Toda corrida se ejecutó con turbo desactivado y con la frecuencia solicitada verificada en cada ventana, con una tolerancia de $\pm 5\%$. Una ventana que no cumple ese criterio no recibe etiqueta. Tras cada cambio de nivel se descartan unos 10 ms, porque un procesador delegado tardaba entre 10 y 11 ms en engancharse al límite. También se descartan 0.15 s al inicio y al final de cada corrida, porque en los kernels de una sola región paralela larga un hilo que termina antes queda ocioso y diluye el promedio de frecuencia. Al restringir solo los

procesadores lógicos delegados, la razón de rendimiento entre el nivel más alto y el más bajo se mantuvo en 0.995. Al restringir también sus hermanos de *SMT*, esa razón subió a 3.78, cercana a la razón de cuatro esperada entre esos niveles.

3.1.1. Composición de fase por kernel

Las familias de cargas, sus variantes y su suite de procedencia se enumeran en el material complementario del repositorio (sección 2.2.1). La Figura 3.1 resume la composición por familia, y el desglose por configuración de kernel queda también en el material complementario. El conjunto cubre ambos regímenes y conserva configuraciones próximas al *ridge*.

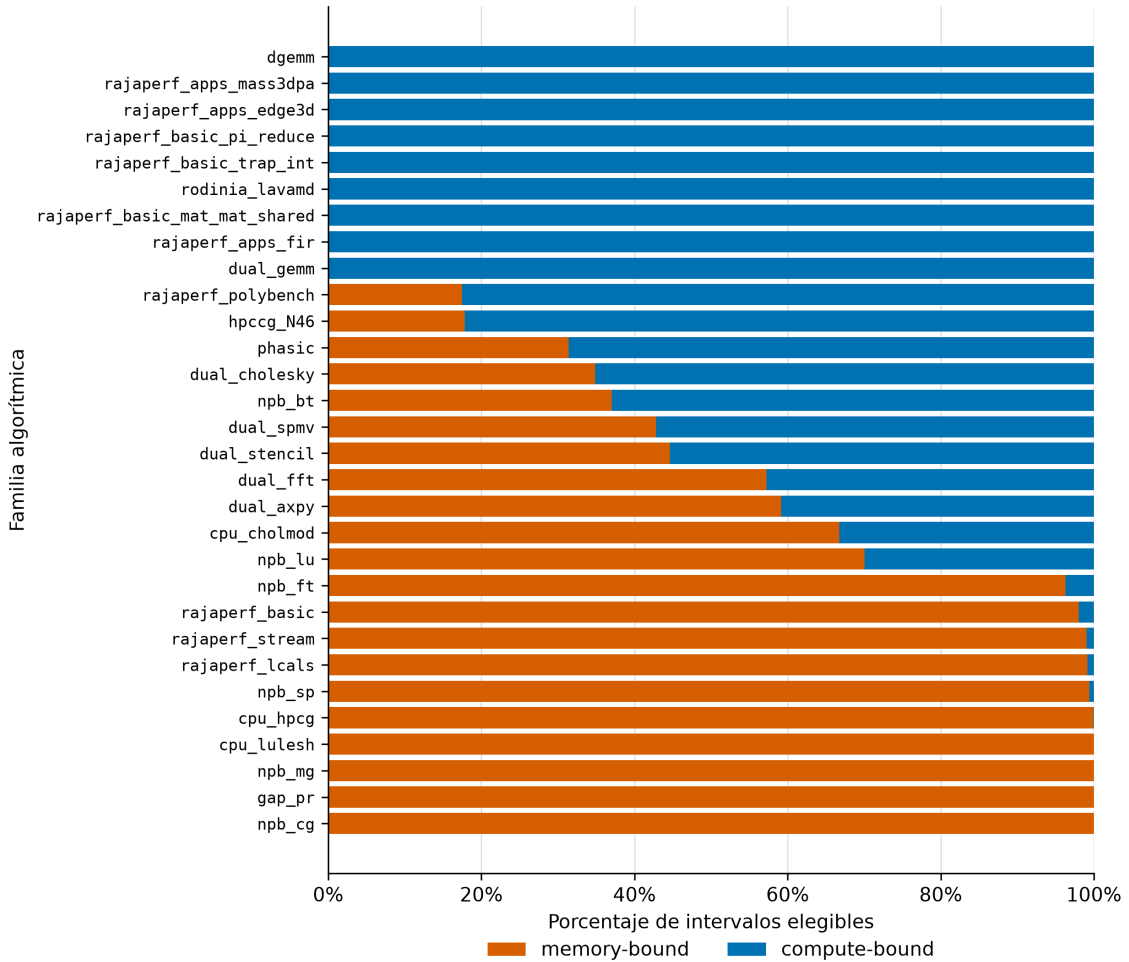


Figura 3.1: Porcentaje de intervalos elegibles clasificados *compute_bound* y *memory_bound* por familia algorítmica. Las variantes de tamaño se agregan dentro de cada familia. Fuente: elaboración propia.

La composición intermedia no se concentra en un solo algoritmo. *npb_bt*, *npb_lu*, *cpu_*

cholmod, *hpccg_cpu_N46*, las variantes *phasic* y los tamaños dirigidos de FFT y Cholesky aportan intervalos de ambos regímenes. Estas configuraciones evitan que el conjunto quede limitado a ejemplos alejados del punto de inflexión.

Tres kernels de patrón de acceso irregular de Rodinia, *kmeans*, *sradi* y *particlefilter*, se midieron por separado y resultaron casi puramente *memory-bound*. No aportaban ejemplos *compute-bound* ni mixtos, y quedaron fuera del conjunto de entrenamiento porque solo añadían redundancia *memory-bound*.

3.1.2. Variación de la composición con la frecuencia

La proporción global de intervalos *compute_bound* aumentó desde 29.0 % en el nivel nativo hasta 46.4 % en F8, el nivel fijo más bajo. La figura 7.1 del material complementario presenta esta variación. Es consistente con un *ridge* que baja junto con el techo de cómputo, así que una parte de los intervalos cercanos a la frontera cambia de etiqueta. La cifra agrega todas las configuraciones, y las cargas no aportan el mismo número de intervalos.

3.1.3. Techos Roofline y *ridge*

Los techos del modelo Roofline se calibraron de forma empírica sobre la propia plataforma (sección 1.1.2). El punto de inflexión desciende con el reloj, de 8.69 FLOP/byte en la referencia nativa a 2.92 FLOP/byte en F8 (Figura 3.2). Baja porque el techo de cómputo baja, mientras el ancho de banda sigue limitado por memoria.

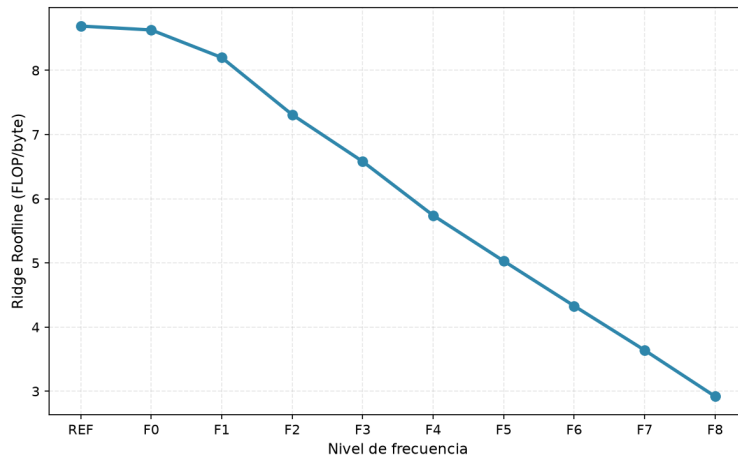


Figura 3.2: *Ridge* Roofline según el nivel de frecuencia CPU. El punto de inflexión desciende de 8.69 FLOP/byte en referencia nativa a 2.92 FLOP/byte en F8. Fuente: elaboración propia.

Validación de los techos. Para verificar la calibración con un mecanismo independiente se usó Intel Advisor, que contabiliza FLOPs y bytes mediante instrumentación binaria y no con contadores de PMU [46]. El ancho de banda calibrado coincide con el de Advisor dentro de 12 a 15 % (58.4 a 58.8 GB/s frente a 64.0 a 66.8 GB/s). El techo de cómputo (417.8 a 555 GFLOP/s) es consistente con la referencia de Advisor, que midió 577.7 GFLOP/s sobre los mismos seis núcleos. El punto de inflexión del nivel nativo queda entre 7.0 y 9.3 FLOP/byte, compatible con los 8.69 FLOP/byte usados en la campaña.

3.1.4. Clasificador de fase en CPU

Los resultados del clasificador siguen el protocolo LOFO y la exactitud balanceada por celda definidos en la sección 2.3.

Muestra de entrenamiento

El modelo CPU se ajustó con 42 458 intervalos de 30 familias (21 350 *compute_bound* y 21 108 *memory_bound*), con un tope de 1000 intervalos por celda de familia y clase (figura 7.2 del material complementario). La sensibilidad a ese tope mantuvo la exactitud entre 0.720 y 0.739 al variarlo de 100 a 30 000, sin tendencia creciente. La unidad de generalización sigue siendo la familia, no cada intervalo.

Variables de entrada

El candidato CPU usa las seis variables definidas en la sección 2.3. La auditoría de correlación (sección 2.3.1) se calculó sobre las doce columnas candidatas (figura 7.3 del material complementario). El par más alto, de 0.89 (Pearson), une MPKI con las referencias de caché por millar de instrucciones, dos normalizaciones de la misma señal de fallos de caché, y de ese par se retuvo MPKI. Sobre las seis variables finales ningún par supera 0.85, y la mayor correlación absoluta es 0.74 (Spearman), negativa, entre IPC y la fracción de ciclos detenidos por memoria. El factor de inflación de la varianza máximo es 5.5 con Pearson y 7.3 con rangos, ambos de IPC, por debajo del umbral de alerta de 10. No fue necesario retirar ninguna variable adicional.

Ampliar el vector de seis a doce entradas redundantes cambió la exactitud balanceada de 0.728 a 0.724 (IC95 de la diferencia, $-0,026$ a $+0,011$). Retirar la frecuencia observada la baja a 0.720, y usarla como única entrada da 0.592. Por eso se conservaron seis variables, incluida la frecuencia. La redundancia perjudicó más a la regresión logística, y a XGBoost no le ofreció una mejora medible de inferencia ni de precisión.

Selección de modelo e hiperparámetros

La Figura 3.3 compara los candidatos con hiperparámetros fijos, cinco semillas y el mismo protocolo LOFO. XGBoost, Extra Trees y Random Forest quedan dentro del mismo intervalo de incertidumbre, y la regresión logística rinde menos. La puntuación F_1 macro agrupada de los modelos de seis variables es de 0.205 para la regla mayoritaria, 0.713 para el árbol de profundidad 1, 0.765 para la regresión logística, 0.717 para Extra Trees, 0.763 para Random Forest y 0.760 para XGBoost. Se seleccionó XGBoost por su menor latencia entre los modelos de árboles. La latencia de una fila y un hilo, medida en el nodo con el proceso fijado a cuatro núcleos, es de 243 μ s de mediana (404 μ s en el percentil 99) para XGBoost, frente a 5.3 ms (5.5 ms) de los dos bosques, más de trece veces mayor. El árbol de profundidad 1 y la regresión logística toman 105 μ s (109) y 450 μ s (475).

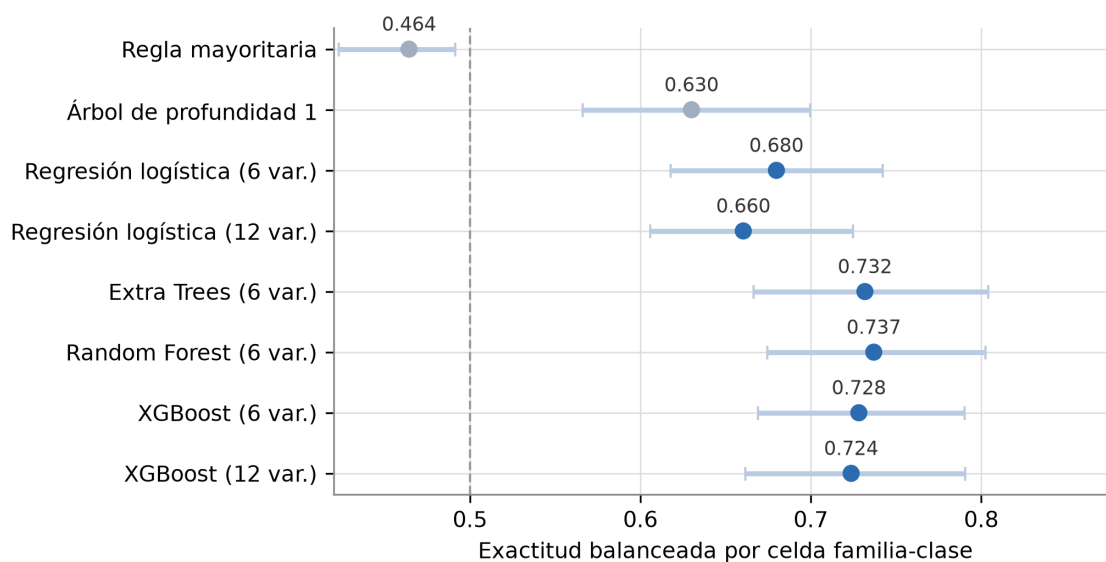


Figura 3.3: Exactitud balanceada por celda de familia y clase de cada modelo, con intervalo de confianza del 95 % obtenido por remuestreo de las 30 familias, con hiperparámetros fijos. Las variantes de doce variables se detallan en el material complementario del repositorio. Los intervalos se solapan entre todos los modelos con capacidad suficiente. Fuente: elaboración propia.

Búsqueda de hiperparámetros. La búsqueda anidada con Optuna [29] no produjo una mejora operativamente útil. Para XGBoost, la diferencia pareada por familia frente a la configuración fija es de +0,019 (IC95 de -0,007 a +0,050, que incluye el cero; 14 de 30 familias mejoran y 8 empeoran). Para la regresión logística es distinguible de cero, pero es irrelevante en magnitud (+0,002). El puntaje de validación interna de Optuna en cada pliegue varía poco (de 0.734 a 0.761 de exactitud balanceada), mientras la exactitud balanceada

sobre la familia externa retenida va de 0.487 a 1.000. Por eso el criterio interno no distingue configuraciones buenas de malas para una familia nueva. Se conservó la configuración fija de XGBoost, la más simple de reproducir. Junto con la comparación de modelos y de vectores de entrada, esto indica que el desempeño no está limitado por la capacidad del modelo ni por sus hiperparámetros.

La configuración congelada es la de la Tabla 3.1.

Tabla 3.1: Configuración final del clasificador CPU.

Elemento	Valor
Modelo	XGBoost, 100 árboles, profundidad máx. 6, objetivo <code>binary:logistic</code>
Entradas	6 variables PMU (sección 2.3)
Ponderación	Peso total igual por celda de familia y clase (ec. (2.3)), con <code>scale_pos_weight = 1</code>
Salida	Probabilidad de <i>memory-bound</i> ; confianza $\geq 0,85$ (<i>revisar</i> por debajo)
Semilla	20260918

Evaluación del modelo final

La métrica principal, la exactitud balanceada por celda, fue 0.728 sin abstención (IC95 de 0.669 a 0.790). La variación entre las 30 familias limita la distinción entre configuraciones cercanas. Esa cifra depende de la unidad de separación, porque mezclar intervalos al azar da 0.874 y dejar fuera un kernel da 0.733. Los valores de las demás métricas de CPU están en la Tabla 3.2 (la Tabla 3.3 de la sección 3.2.3 trae las de GPU, para comparar). Sobre los 1 309 755 intervalos elegibles, el modelo alcanzó una F1 macro agrupada de 0.760 y una exactitud global de 0.769, con sensibilidad de 0.714 para *compute-bound* y de 0.806 para *memory-bound*. La Figura 3.4 muestra las matrices de confusión.

Tabla 3.2: Métricas del clasificador CPU sin abstención [47, 48], media de cinco semillas de muestreo. El detalle con desviaciones e intervalos está en el material complementario.

Métrica	Valor
Exactitud	0.769
Sensibilidad <i>compute-bound</i>	0.714
Sensibilidad <i>memory-bound</i>	0.806
F1 macro agrupada	0.760
AUC-ROC agrupada	0.842
Coefficiente de Matthews	0.520
Exactitud balanceada por celda	0.728

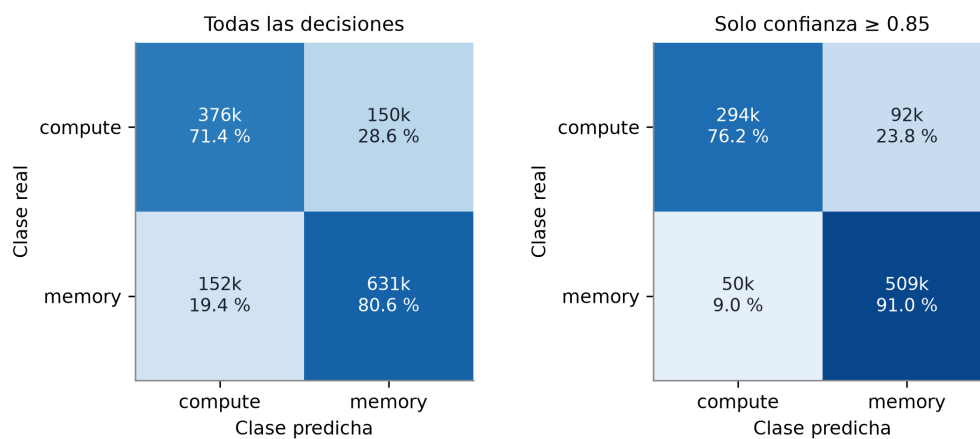


Figura 3.4: Matrices de confusión del modelo final. A la izquierda, cuando se obliga a emitir una clase en todos los intervalos; a la derecha, solo sobre los intervalos con confianza mayor o igual a 0.85. Fuente: elaboración propia.

El resultado no es uniforme entre familias (Figura 3.5). Catorce familias superan 0.9 de exactitud y cinco quedan por debajo de 0.55, con *npb_ft* en 0.314. Las mejores tienen un comportamiento extremo y estable. Las peores tienen un perfil de contadores que coincide con el de familias de la clase contraria.

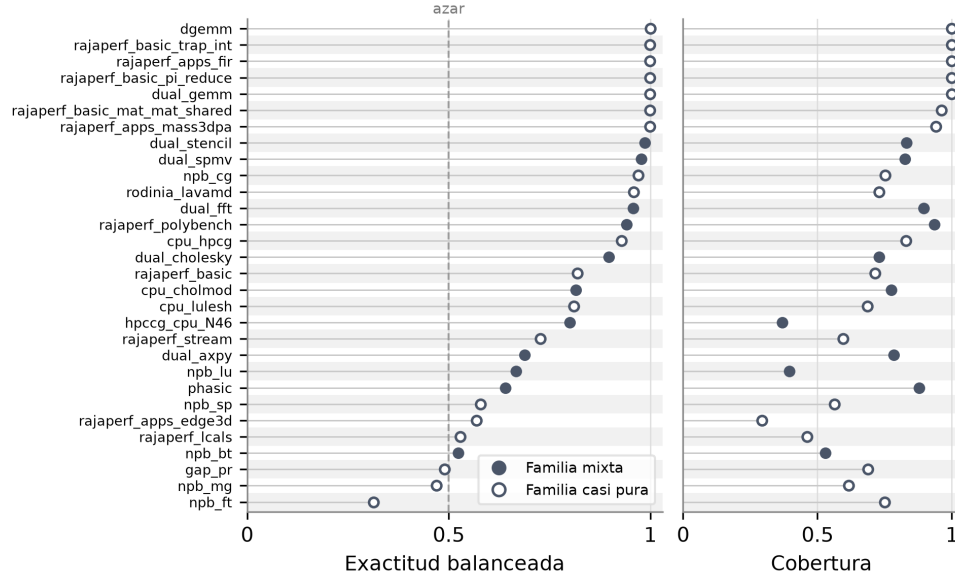


Figura 3.5: Exactitud balanceada (izquierda) y cobertura de la decisión automática (derecha) para cada familia retirada, ordenadas por exactitud. El marcador lleno distingue las familias mixtas, entre 10 y 90 % de intervalos *memory-bound*, y el hueco las casi puras; la línea discontinua marca el azar. Fuente: elaboración propia.

Al bajar el reloj, la exactitud balanceada por celda y la cobertura de la decisión automática caen juntas (figura 7.4 del material complementario). Es consistente con el desplazamiento del *ridge*. En los niveles bajos, más intervalos pasan a la clase de cómputo, quedan cerca de la frontera y el modelo se abstiene más. La exactitud global repunta en F8 (0.757, frente a 0.710 en F7), pero es un efecto de composición. F8 aporta más intervalos que los demás niveles (256 531, frente a 85 447 a 186 309), en su mayoría *compute-bound*, y la exactitud balanceada por celda no rebota.

Decisión selectiva y costo de inferencia

Se congeló el umbral de abstención CPU en 0.85 (Figura 3.6, derecha). Elevarlo a 0.90 aumentaba la exactitud de selección solo 0.003 y reducía la cobertura en 6.5 puntos. Es una decisión de diseño posterior al resultado. Elegir el umbral por pliegue, dentro de cada familia retirada, da prácticamente lo mismo (0.734 sobre lo decidido), así que congelar un único valor no infla el resultado. Con ese umbral, sobre el modelo final, el agente decide el 72.2 % de los intervalos (75.1 % promediando por familia y 29.3 % en la familia peor cubierta), y el resto recibe *revisar*. Sobre lo que decide, la exactitud balanceada por celda sube de 0.728 a 0.738, la F1 macro agrupada de 0.760 a 0.842, y la sensibilidad de las clases *compute-bound* y *memory-bound* de 0.714 y 0.806 a 0.762 y 0.910 (Figura 3.4). La ganancia por celda es modesta y la mayor parte de la mejora aparece en las métricas agrupadas. Abstenerse al

azar en una proporción equivalente de intervalos no mejora nada (0.716 y 0.752). Por eso la ganancia viene de que la confianza identifica casos difíciles, y no de decidir menos (Figura 3.6, derecha).

El límite de la abstención es que la confianza está mal calibrada fuera de la familia de entrenamiento (Figura 3.6, izquierda). El modelo declara una confianza media de 0.890 y acierta con 0.764 de exactitud (error de calibración esperado de 0.126, con una sola semilla, frente a 0.769 de exactitud global promediando cinco). Además, hay familias que fallan con confianza alta, a las que el umbral no protege. Entre los intervalos con confianza mayor o igual a 0.9, *npb_ft* acierta el 32 % y *npb_mg* el 35 %.

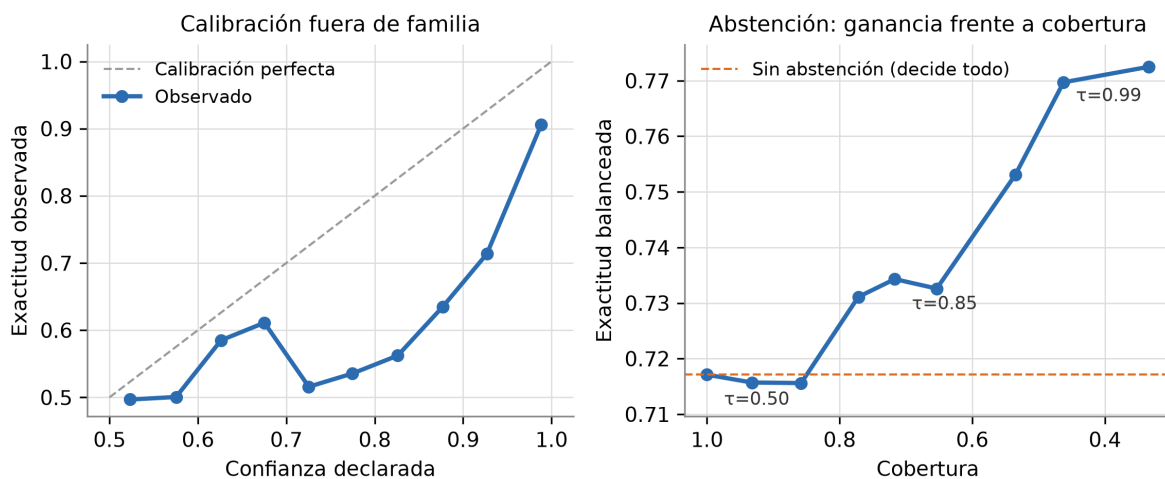


Figura 3.6: A la izquierda, confianza declarada frente a exactitud observada sobre familias no vistas; la curva por debajo de la diagonal indica sobreconfianza. A la derecha, exactitud balanceada sobre lo decidido en función de la cobertura al variar el umbral τ (el eje de cobertura está invertido: a la derecha, el umbral es más estricto); la línea discontinua naranja es la exactitud sin abstención; ambas curvas usan una sola semilla de muestreo, por lo que sus valores difieren ligeramente de los promedios de cinco semillas que se dan en el texto. Fuente: elaboración propia.

Ante la salida *revisar*, el agente conserva la última frecuencia aplicada. Las latencias de inferencia de la selección de modelo incluyen la sobrecarga del intérprete de Python, y son una cota superior para la implementación integrada en el agente (sección 3.3.2).

Dónde está el límite del resultado

Cuatro análisis indican que el límite proviene de la información contenida en las seis variables de entrada y no del modelo.

El primero es que un clasificador sin ajuste, de 25 vecinos más cercanos sobre cuatro tasas estandarizadas, alcanza 0.684 con el mismo protocolo, y 0.699 si recibe además la frecuencia. Queda a menos de 0.045 del modelo final. El segundo es la existencia de perfiles gemelos entre familias de clase opuesta (Figura 3.7). El caso más simétrico enfrenta a *gap_pr*, *memory-bound*, con la fase *compute-bound* de *phasic*. La primera tiene MPKI de 0.82, IPC de 1.11 y tasa de fallos de 0.76, y la segunda tiene MPKI de 0.94, IPC de 0.97 y tasa de fallos de 0.72. El par más cercano en distancia estandarizada es otro. La celda rara *memory-bound* de *rajaperf_basic_mat_mat_shared* (0.1 % de sus intervalos; la familia es *compute-bound*) queda a la mitad de distancia de la celda *compute-bound* típica de *rajaperf_basic_pi_reduce*, de la misma suite. Ningún modelo que reciba solo esas señales puede separar estos pares, y las tres familias involucradas están entre las de peor resultado.

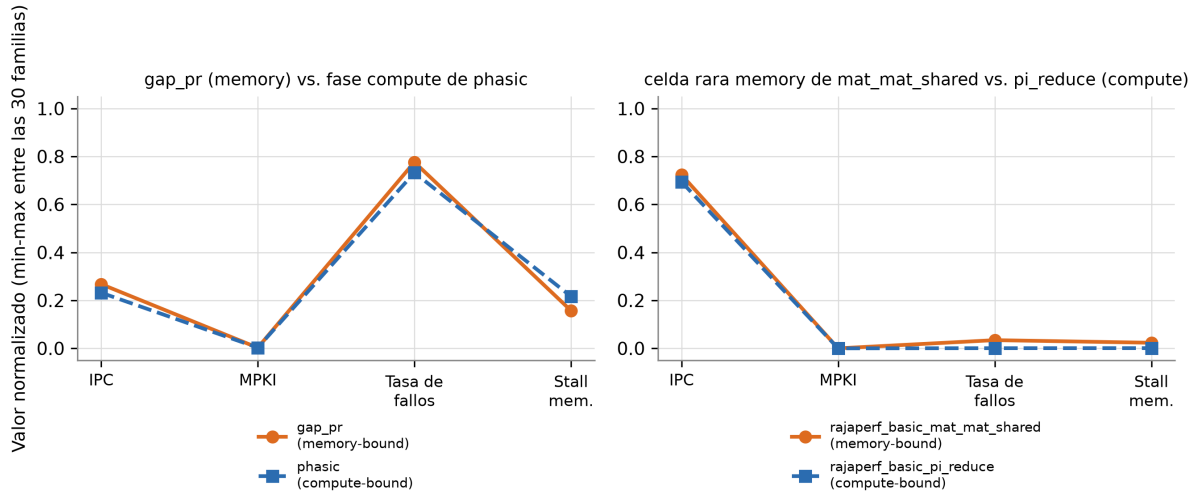


Figura 3.7: Cuatro variables de entrada, normalizadas al rango de las 30 familias, para los dos pares de celdas más confusables entre clases opuestas. Fuente: elaboración propia.

El tercero es la posición respecto del *ridge*. Los intervalos a menos de un factor dos del punto de inflexión son el 12.5 % del total y se clasifican con 0.521 de exactitud, frente a 0.799 en el resto. Sin embargo, concentran solo el 25.3 % de los errores, así que tres cuartas partes ocurren lejos de la frontera, donde la etiqueta no es ambigua. El error tampoco es simétrico. En la franja *compute-bound* inmediata al *ridge* la exactitud queda por debajo del azar, y en la franja simétrica *memory-bound* es bastante mayor. Es consistente con el déficit de sensibilidad de *compute-bound* (sección 3.1.4) y con los perfiles gemelos (figura 7.5 del material complementario).

El cuarto es la curva de aprendizaje por número de familias (figura 7.6 del material complementario). Al ajustar con subconjuntos de 3 a 23 familias, el resultado sobre la familia

retirada sube de 0.615 a 0.718, unos 0.037 por cada duplicación del catálogo. Es consistente con que la diversidad algorítmica gobierne el resultado, aunque el diseño no separa el número de familias del tamaño de la muestra. Dentro del alcance de este trabajo, la mejora disponible por esa vía es pequeña.

Como referencia de límite superior, el mismo modelo con la intensidad operacional medida como entrada adicional alcanza 0.9989, frente a 0.728 sin ella. No es una mejora del modelo, porque la intensidad operacional es la cantidad con la que se define la etiqueta, y por eso queda fuera del vector de entrada. Solo muestra que la brecha entre 0.728 y 1 es información que las seis variables del agente no transportan. Los contadores de FLOPs y de bytes de DRAM sí la contienen, pero no se pueden usar durante la operación del agente (sección 2.3.1).

3.1.5. Invarianza del clasificador de CPU frente a la acción del agente

La frecuencia observada es una de las seis variables del clasificador de CPU y el agente la modifica. Por eso se le aplicó la prueba contrafactual de la sección 2.3.4, que fuerza la frecuencia a un valor fijo y mide cuántas predicciones cambian. Sobre el modelo final y sus 1.31 millones de intervalos de 49 kernels, forzar la frecuencia a 3.2, 2.9, 2.0 y 0.8 GHz cambia, respectivamente, el 10.6, 10.1, 4.5 y 8.8 % de las predicciones. Existe entonces una dependencia moderada. Pero el cambio que el agente ejecuta es acotado, de F0 (3.2 GHz) a F1 (2.9 GHz), y sobre las 179 774 filas observadas cerca de 3.2 GHz solo cambia el 0.42 % de las predicciones. Con ese espacio de acción, la retroalimentación por esta variable es despreciable, y el modelo de CPU se despliega sin modificaciones.

3.1.6. Política de frecuencia por clase en CPU

Se comparó el EDP de cada nivel fijo frente a la base conforme al procedimiento de la sección 2.3. La política resultante es *no actuar* en ambas clases (Figura 3.8). La figura 7.7 del material complementario separa la energía y el tiempo que producen ese resultado. En cómputo, F0 mejora 0.56 % frente a la base, menos que el efecto mínimo relevante de 1 %, y todos los niveles inferiores empeoran el EDP. En memoria tampoco hay ganancia agregada a menor reloj. F1 empeora 5.4 % (IC95 $[-6, 8; -3, 4]$) y los niveles inferiores empeoran más. Algunos kernels STREAM presentan ahorros puntuales entre 4 y 11 % en su mejor nivel, pero el signo y la magnitud varían entre niveles y repeticiones, y no justifican una regla común.

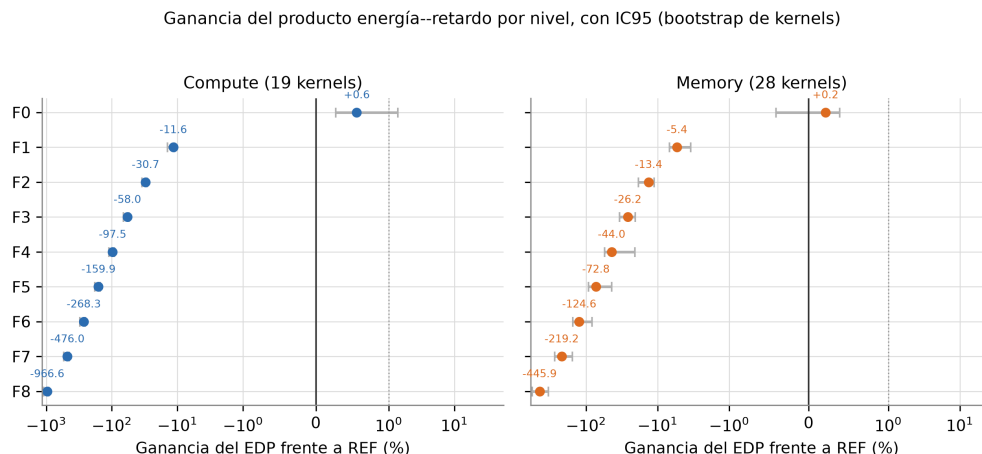


Figura 3.8: Ganancia del EDP frente a la base por nivel y clase (cómputo: 19 kernels; memoria: 28), con IC95 por *bootstrap* de kernels; eje simétrico-logarítmico. Valores p y límites numéricos en el material complementario. Fuente: elaboración propia.

La energía sube al bajar la frecuencia porque el tiempo crece más de lo que cae la potencia (figura 7.7 del material complementario). En memoria se mantiene cerca de la referencia hasta unos 2 GHz. La razón es que unos 85 W, el 84 % de la potencia a 3.2 GHz, se consumen sea cual sea la frecuencia (Figura 3.9). Reducir la frecuencia a 0.8 GHz ahorra solo 15 % de potencia mientras alarga la ejecución. Incluso un oráculo que eligiera el mejor nivel medido por kernel ganaría 2.2 % en promedio, y 22 de 47 kernels quedarían por debajo del efecto mínimo de 1 % (los 49 kernels del clasificador, sin los tres *phasic* de duración fija, y con *ptrchase*, que solo se midió para esta política). Esta cota es optimista, porque el mejor nivel también captura ruido de medición.

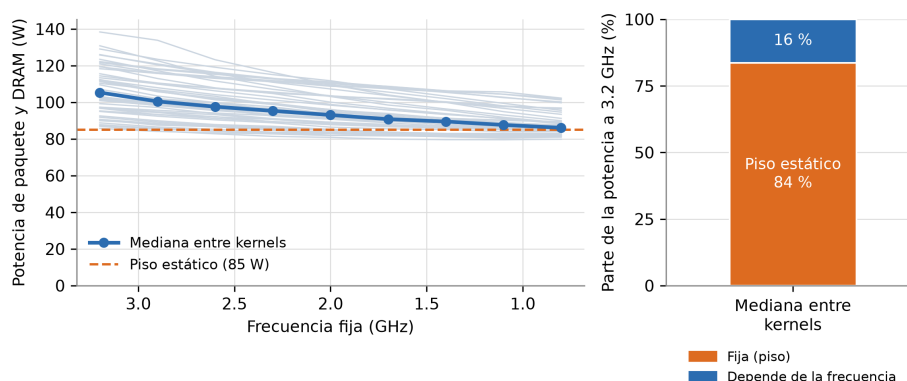


Figura 3.9: A la izquierda, potencia media de paquete y DRAM de cada kernel (líneas grises) y su mediana en cada nivel fijo, con el piso estático ajustado. A la derecha, parte de la potencia a 3.2 GHz que corresponde al piso: la barra muestra la mediana entre kernels (84 %), y según el kernel va de 62 a 94 %. Fuente: elaboración propia.

Tamaño de efecto detectable. Un resultado de *no actuar* solo es informativo si se conoce el efecto que el experimento podía detectar. La variabilidad tiene dos fuentes. El error de medición es pequeño, porque repetir el mismo kernel en el mismo nivel da un producto energía-retardo con una desviación estándar del logaritmo de 1.05 % en mediana. La diferencia entre familias es mayor. La dispersión entre las 19 familias *memory_bound* es de 5.35 % en F1 y 9.70 % en F2, entre cinco y diez veces la primera. Por eso repetir más las mediciones no reduciría la incertidumbre de la ganancia promedio, y solo lo haría un catálogo con más familias. Un modelo de potencia $P(f) = P_0 + c f^k$, ajustado a cada kernel con un error mediano de 0.34 W y un piso casi idéntico entre kernels (80 a 100 W), muestra que bajar la frecuencia mejora el EDP únicamente si la pendiente α de $\log T$ frente a $\log f$ es menor que un valor crítico. Ese valor tiene mediana 0.14 y en ningún kernel supera 0.33, mientras que los kernels de cómputo denso tienen un α cercano a 1. El oráculo que elige el mejor nivel de cada kernel gana como máximo 12.5 %. Decidir actuar exige entonces identificar la región de α bajo, y la clase Roofline por sí sola no la determina.

3.2. Resultados de GPU

3.2.1. Validación de control y etiquetado

El actuador de GPU sostuvo el bloqueo del reloj de *streaming multiprocessors* bajo carga. Con objetivos de 600 y 1200 MHz se registraron 305 y 174 lecturas activas coincidentes con el valor solicitado, y la restauración se verificó al finalizar las campañas dedicadas (la premisa física de la política se comprueba en la sección 3.3.1). En el nivel de referencia, los puntos de inflexión medidos fueron 14.97 FLOP/byte para FP32 y 6.90 FLOP/byte para FP64. La caracterización solicitó ambas precisiones a la vez y mostró que la precisión declarada no siempre es la ejecutada. *rodinia_backprop* y *rodinia_myocyte*, declarados FP32, ejecutaron 82.3 y 71.3 % de sus FLOPs en FP64, y se conservaron con precisión FP64 porque su clasificación *memory-bound* no cambia con ninguno de los dos techos. *rodinia_hotspot*, con 30.4 % en FP64 y una intensidad de 7.21 FLOP/byte en el borde del *ridge* FP32 (7.28 a 7.98), se retiró del catálogo por no admitir un único techo.

3.2.2. Conjunto de datos y clasificador de fase en GPU

El conjunto de entrenamiento del clasificador GPU reúne 483 corridas aceptadas de 19 kernels y 16 familias algorítmicas, en los diez niveles de frecuencia (referencia nativa y F0 a F8). Proceden de la campaña GPU final y de tres campañas adicionales de *kmeans*, *miniBUDE* y *conv2d*. Cada corrida tiene una etiqueta Roofline obtenida fuera de línea y

señales NVML agregadas por corrida. Los tres núcleos CUDA de RAJAPerf (GEMM, *stencil* de calor y Jacobi 2D) forman familias separadas, porque sus perfiles Roofline difieren entre sí. La composición es de 171 corridas *compute_bound* y 312 *memory_bound*, y 13 de las 16 familias son homogéneas respecto de su etiqueta.

Los candidatos reciben cinco señales agregadas por corrida, que son las medianas de utilización GPU, utilización de memoria, potencia y reloj SM, más la desviación estándar de la utilización de memoria. Esta última evita que la mediana diluya cargas de lanzamientos cortos y repetidos como *kmeans*, de intensidad operacional alta pero con una firma NVML agregada similar a la de una carga poco ocupada. Las cinco señales no presentan colinealidad relevante. El factor de inflación de varianza (VIF) fue de 1.89 para la utilización GPU, 1.76 para la de memoria, 2.26 para la potencia, 1.37 para el reloj SM y 1.32 para la desviación estándar de la utilización de memoria, todos muy por debajo del umbral de alerta de 10 del contrato de variables.

Con el protocolo de validación definido en la sección 2.3, ninguno de los siete modelos candidatos domina de forma concluyente en exactitud balanceada (Figura 3.10). *Random forest* (0.807), *extra trees* (0.806) y la regresión logística (0.792) resultaron estadísticamente indistinguibles entre sí por remuestreo pareado de familias, con probabilidades de superioridad, la fracción de réplicas *bootstrap* en que un modelo supera al otro, entre 0.51 y 0.59. *Random forest* sí superó con evidencia razonable a XGBoost, con probabilidad de superioridad de 0.984. La búsqueda anidada de hiperparámetros empeoró cuatro de los cinco modelos ajustables, por ejemplo *random forest* pasó de 0.807 a 0.778, y solo mejoró el árbol de profundidad 6, de 0.727 a 0.783. Por eso se descartó.

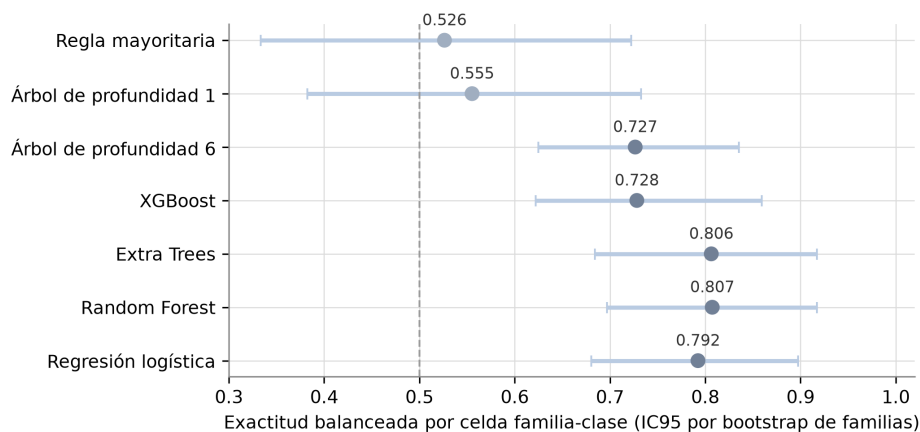


Figura 3.10: Exactitud balanceada por celda familia-clase de los siete modelos candidatos con las cinco señales, con intervalo de confianza del 95 % por *bootstrap* de familias. Fuente: elaboración propia.

El núcleo GEMM de RAJAPerf y *LUD* de Rodinia no se resuelven con ninguna variable admisible, por una razón física. Cambian de régimen Roofline según el nivel de frecuencia, porque el punto de inflexión se desplaza con el reloj mientras la intensidad operacional del kernel es fija (*LUD* es *memory_bound* en los relojes más altos y *compute_bound* en los más bajos). Distinguirlos exigiría una variable de distancia al *ridge*, y esa variable sería fuga de la propia etiqueta.

3.2.3. Candidato operativo de GPU: invarianza frente a la acción del agente

El agente fija el reloj de la GPU, y el reloj y la potencia entre las señales candidatas dependen de esa acción. Por eso la selección exige entradas invariantes (sección 2.3.4). La auditoría, sobre las mismas 483 corridas y sin mediciones nuevas, rompe el empate entre los tres primeros modelos (Figura 3.11). Al fijar de forma contrafactual el reloj de cada observación en 1410 MHz, la regresión logística (0.792 con las cinco señales, ECE 0.0569) cambia de clase el 26.7% de sus predicciones, y su exactitud baja a 0.654 al excluir el reloj. El *random forest* conserva el desempeño al retirar las entradas afectadas por la actuación, con 0.807 de exactitud balanceada usando cinco variables, 0.812 sin reloj y 0.819 sin reloj ni potencia, sin diferencia distinguible entre las tres variantes. El modelo desplegado es, por tanto, el *random forest* de tres señales invariantes (utilización de GPU, utilización de memoria y desviación estándar de esta última), elegido por invarianza y no por exactitud. La regresión logística de cinco señales queda como referencia estática.

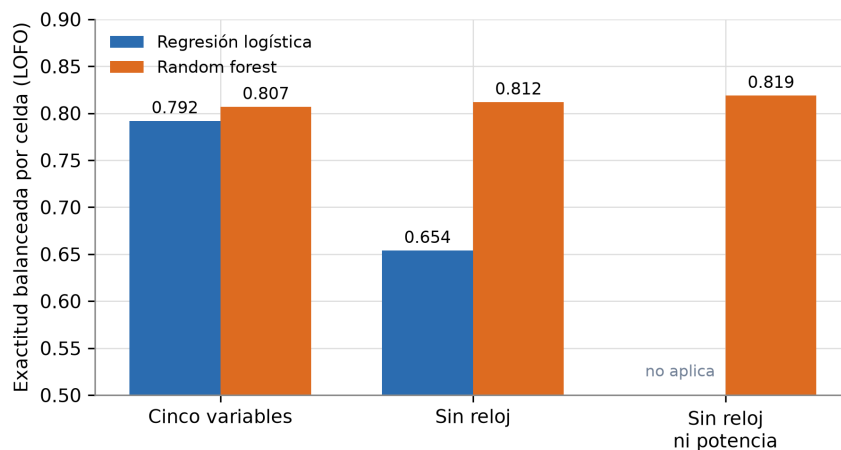


Figura 3.11: Exactitud balanceada por celda (LOFO, 483 corridas, 16 familias) al excluir las variables que dependen de la acción del agente. Fuente: elaboración propia.

Al evaluar cada familia con un modelo entrenado sin ella, el *random forest* clasifica co-

rectamente el 89.2% de las corridas, con 0.819 de exactitud balanceada por celda (Tabla 3.3). Reconoce el 82.5% de las *compute_bound* y el 92.9% de las *memory_bound* (Figura 3.12, izquierda). El resultado no es parejo entre familias (figura 7.8 del material complementario). Diez de las dieciséis se clasifican sin error, mientras *rodinia_gaussian* (0.50, la familia más cercana al *ridge* del catálogo), *dual_cholesky* (0.60) y *rodinia_lud* (0.60, una de las que cambian de régimen con la frecuencia) quedan en el nivel del azar o por debajo.

Tabla 3.3: Métricas del clasificador GPU sin abstención [47, 48], validación por familia retirada sobre 483 corridas y 16 familias. El detalle con intervalos está en el material complementario.

Métrica	Valor
Exactitud	0.892
Sensibilidad <i>compute-bound</i>	0.825
Sensibilidad <i>memory-bound</i>	0.929
F1 macro agrupada	0.881
AUC-ROC agrupada	0.913
Coefficiente de Matthews	0.763
Exactitud balanceada por celda	0.819

Su ECE con validación por familia es 0.0868, calculado sobre diez rangos de confianza de igual amplitud. La confianza es fiable donde más pesa. Las 263 corridas con confianza superior a 0.95 se aciertan el 95.4%, mientras que los rangos intermedios agrupan pocas corridas de una o dos familias y la exactitud oscila. El mínimo, en 0.75 a 0.80, son sobre todo corridas de *rodinia_gaussian* (figura 7.9 del material complementario, izquierda).

Con el umbral de 0.90 alcanza 0.900 de exactitud balanceada por celda, con una cobertura media por celda de 65.7% (62.1% de las corridas; figura 7.9 del material complementario, derecha). Sobre las 296 corridas que decide, no comete errores en *memory_bound* (224 aciertos), y sus 15 errores son *compute_bound* clasificadas como memoria (Figura 3.12, derecha). La abstención recae sobre todo en las familias de cómputo, ya que decide el 42% de las *compute_bound* y el 72% de las *memory_bound*. El modelo se ajustó sobre las 483 corridas después de cerrar la evaluación LOFO. La inferencia de una fila tarda 4.7 ms en mediana y 5.0 ms en el percentil 99, medida en Python con *scikit-learn* y no dentro del agente, frente a los segundos que toma detectar y actuar sobre una fase. El agente decide una sola vez por fase, así que una abstención no se reevalúa.

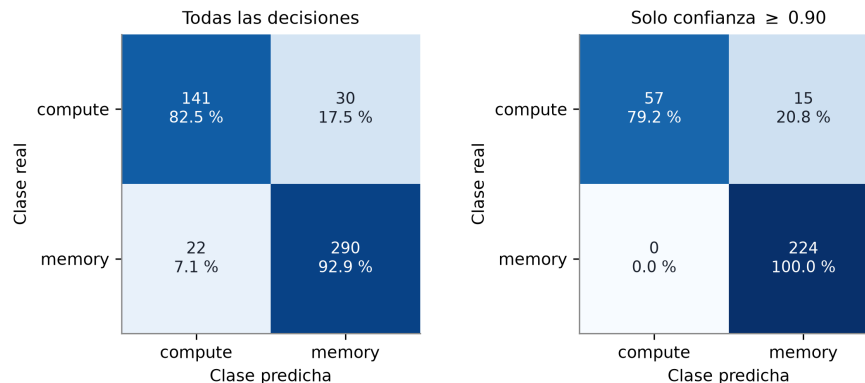


Figura 3.12: Matrices de confusión del modelo desplegado bajo validación por familia (483 corridas, una semilla; las métricas del texto por celda son promedios de cinco semillas). A la izquierda, todas las decisiones; a la derecha, solo las de confianza mayor o igual a 0.90. Fuente: elaboración propia.

La importancia por permutación (figura 7.10 del material complementario) muestra que la utilización de memoria y su variabilidad concentran la dependencia predictiva del *random forest* desplegado. La utilización global de GPU aporta una señal menor.

3.2.4. Política de frecuencia por clase en GPU

La política GPU se derivó sobre las mismas 483 corridas, conforme al procedimiento de la sección 2.3. El análisis usa la familia como unidad de agregación para que las variantes de tamaño de un mismo algoritmo no pesen varias veces en la decisión. La comparación es pareada con la base, así que cinco de los 19 kernels no entran. `minibude_cuda_bm1` solo tiene corridas válidas en la base, porque en los demás niveles el reloj observado no coincidió con el solicitado. Las variantes pequeñas de `dual_axpy` y `dual_stencil`, el GEMM y el *stencil* de calor de RAJAPerf no tienen corridas aceptadas en la base, ya que son más cortas a relojes altos y no alcanzaron la fracción mínima de muestras NVML posteriores al arranque. Quedan 5 kernels *compute_bound* y 9 *memory_bound*, agrupados en 5 y 8 familias (los dos tamaños de `dual_spmv` forman una sola).

Tabla 3.4: Síntesis de la política GPU. Las ganancias son de EDP frente a la base; la rejilla completa está en la Figura 3.13.

Clase	Evidencia decisiva	Política
Cómputo	F1: +2,1 % (IC95 $[-12,7; 14,2]$, $p = 0,438$); sin mejora confiable	No actuar
Memoria	F1: +8,9 % (IC95 $[3,0; 13,3]$, $p = 0,020$); siete de ocho familias mejoran	Fijar F1

El resultado es **no actuar** en *compute_bound* y **actuar en F1** para *memory_bound* (Tabla 3.4 y Figura 3.13). En *memory_bound*, la validación por familia retirada conserva la dirección en siete de ocho pliegues, con ganancias de 4.2 a 14.8 %, y arroja una ganancia fuera de muestra de 6.5 % (IC95 de $-4,0$ a $13,1$ %). La excepción es *rodinia_gaussian*, que pierde 27.3 %. Es la familia más cercana al *ridge* del catálogo, y por eso su tiempo responde al reloj como el de una carga de cómputo pese a su etiqueta de memoria. Se dejó en el análisis, porque quitarla después de ver la pérdida sesgaría la política a su favor. F2 alcanza una ganancia de 7.4 %, pero no es significativa al usar la familia como unidad.

El mecanismo es el mismo que en CPU. Sobre los 18 kernels con al menos tres niveles de reloj medidos (se excluye *minibude_cuda_bm1*, que solo tiene corridas en la base), los *compute_bound* tienen α mediano de 0.837 (rango 0.344–1.216, $n = 6$) y los *memory_bound* de 0.189 (rango 0.093–0.941, $n = 12$). Su tiempo apenas responde al reloj SM porque el cuello de botella, el ancho de banda de la memoria del dispositivo, pertenece a un dominio de reloj que la matriz no varía. La excepción es *rodinia_gaussian*, con $\alpha = 0,941$ pese a ser *memory_bound* sin ambigüedad, y por eso es la única familia que pierde en la validación por familia retirada. *rodinia_lud*, con el menor margen de clase (0.60), gana 13.9 %. Al clasificador lo confunde un kernel cuya etiqueta cambia con el nivel, y a la política, uno con etiqueta estable pero con respuesta al reloj parecida a la de la otra clase. Ninguna de las dos familias se excluyó, porque hacerlo seleccionaría datos por el resultado que producen.

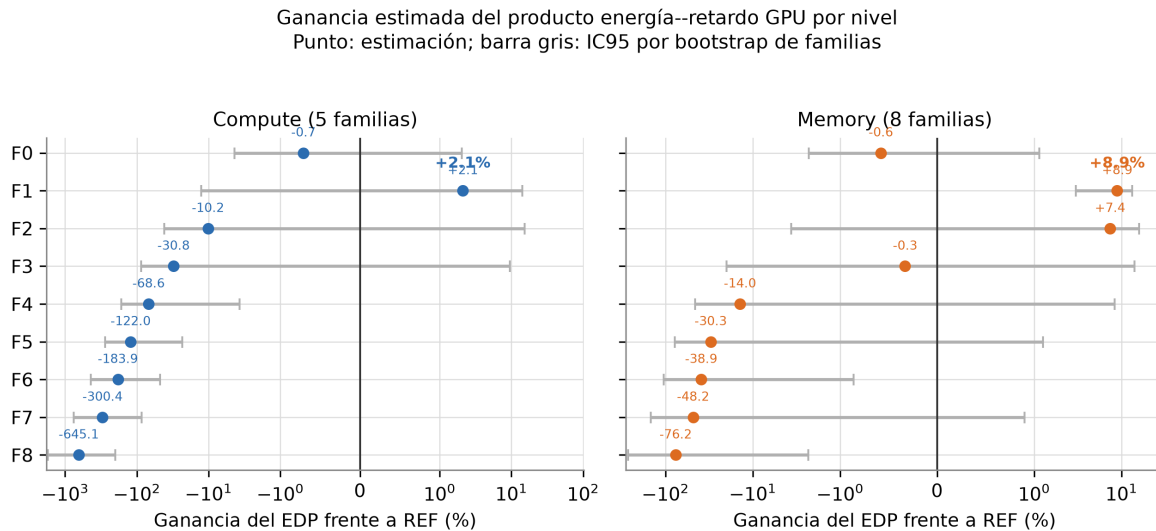


Figura 3.13: Ganancia del EDP GPU frente a la base por nivel y clase, con la familia como unidad (cómputo: 5 familias; memoria: 8). Cada punto es la ganancia estimada (en %, con su valor rotulado) y la barra gris, su IC95 por *bootstrap*, cuyos límites numéricos están en el repositorio; el rótulo en negrita señala F1. Valor p de la prueba pareada: en cómputo entre 0.438 y 1.000; en memoria, 0.020 en F1, 0.156 en F2 y entre 0.422 y 0.926 en los demás niveles. El eje horizontal es simétrico-logarítmico para mostrar a la vez F1, cercano a cero, y F8, con pérdidas de más de un orden de magnitud en *compute_bound*. Fuente: elaboración propia.

En *compute_bound*, el promedio descriptivo de F1 (+2.1 %) no es una mejora confiable (Tabla 3.4). La validación LOFO de la regla que escoge F1 produce una ganancia realizada de -4,8 % (IC95 de -13,4 a 0 %), y aplicada a una familia que no participó en elegir el nivel, empeora en promedio. Por eso el daemon conserva la base en *compute_bound*, y la conclusión coincide con la de CPU.

La figura 7.11 del material complementario hace visible el contraste entre clases. En *compute_bound* el tiempo y el EDP crecen juntos desde F1, porque la energía apenas cae. En *memory_bound* el tiempo crece más despacio y la energía cae de forma sostenida hasta F3, así que el EDP se mantiene por debajo de 1 en F1–F2. Después la caída de energía se agota, y a partir de F5 el tiempo domina.

La diferencia viene del piso estático de potencia. Se ajustó $P(f) = P_0 + c f^k$ a la potencia de cada kernel frente al reloj SM y se conservaron los 12 kernels donde el ajuste es confiable. Los otros seis se descartaron por tener menos de siete niveles con datos o $R^2 < 0,8$. El piso P_0 resulta de 38.5 W en mediana y es el 55.8 % de la potencia al reloj más alto (figura 7.12 del material complementario). Es una fracción menor que el 84 % de CPU, así que queda más potencia dinámica por recortar. Es consistente con que *memory_bound* encuentre una

región de mejora en GPU que CPU no encontró en ninguna clase.

3.3. Resultados del agente de control

Antes de esa verificación se comprobó que la exportación a ONNX no altera los modelos. En CPU, el XGBoost exportado se comparó con el original sobre los 1 309 755 intervalos elegibles. La mayor diferencia de probabilidad fue de 3×10^{-7} y ninguna etiqueta cambió. En GPU, el *random forest* exportado se comparó sobre las 483 corridas, con una diferencia máxima de 9×10^{-8} y sin cambios de etiqueta. Ambas están muy por debajo de la tolerancia de 10^{-4} , así que lo que decide el agente en C++ coincide con lo que se validó en Python.

3.3.1. Candado de reloj de GPU y premisa física de la política

Con una carga sintética de fases controladas se comprobó que, al fijar el reloj con `nvidia-smi -lgc`, el reloj observado coincide con el solicitado en los seis valores probados y que el rendimiento responde con la forma que supone la política. El de una carga *compute_bound* escala aproximadamente con el reloj, mientras que el de una carga *memory_bound* no cae hasta llegar a 510 MHz. Es la premisa física de bajar el reloj en la clase *memory_bound* sin costo de tiempo, medida sobre el mismo instrumento que aplicará el agente.

3.3.2. Latencia, restauración y detección del agente

Latencias de conmutación y sobrecarga. Fijar el reloj de GPU tarda unos 50 ms, y el reloj observado alcanza el valor pedido a los 70 a 80 ms. Un cambio completo, con el asentamiento de las lecturas de NVML, cuesta entre 0.35 y 0.37 s. Con la GPU en reposo, el proceso usa entre 0.0 y 0.1 % de un núcleo. En CPU, escribir los límites en los núcleos delegados toma 27.9 ms en secuencia y 2.85 ms en paralelo, y la inferencia del clasificador toma 16.4 μ s en la mediana y 19.3 μ s en el percentil 99. La permanencia mínima de GPU resultante, diez veces esa latencia, fue de 3.7 s. La sobrecarga energética del brazo *sombra* se cuantifica en la Fase 4.

Restauración y coordinación. En cada prueba de terminación anómala inducida (SIGTERM o SIGINT) el agente atendió la señal y devolvió el dispositivo a su estado inicial, con código de salida 0. En GPU el reloj volvió al valor nativo y en CPU el turbo y el rango de frecuencia quedaron idénticos al estado inicial, también cuando ambos procesos corrían a la vez. La prueba de coexistencia ejecutó ambos procesos en brazo activo sobre la aplicación compuesta

A de GPU. El piso de CPU de 3.2 GHz se respetó en el 99.2 % de las muestras, y las cuatro restantes (de unos 0.24 s cada una) están en los flancos de subida.

Clasificación de fase de GPU sobre aplicaciones compuestas. Con el umbral de 0.90, en la aplicación A (tres ciclos) cada brazo tuvo tres decisiones correctas, ninguna errónea y dos abstenciones. Las correctas tuvieron confianza 1.00 y las abstenciones, entre 0.57 y 0.67. En la B (dos ciclos, brazo sombra) hubo dos correctas y dos abstenciones, y *LavaMD*, una familia inédita, se clasificó correctamente con confianza 1.00. Con conteos tan pequeños, la exactitud se mide en la Fase 4.

3.4. Evaluación del agente frente a la base

Cada razón divide la mediana de una métrica del brazo evaluado entre la mediana de esa misma métrica en el brazo base, sobre la misma combinación experimental de la matriz descrita en la sección 2.5. Las métricas son la duración, la energía de CPU, la energía de GPU y el EDP del nodo. Un valor de 1 significa que el brazo iguala a la base. Un valor menor que 1 indica una mejora, y uno mayor que 1, un empeoramiento. Salvo que se indique otra cosa, la base usa el gobernador *performance*; las mediciones con *powersave* se comparan con la base de ese mismo gobernador.

3.4.1. Aplicaciones compuestas

La Figura 3.14 presenta las aplicaciones compuestas, la aplicación A de familias vistas y la B de familias inéditas. Cada bloque de la figura es una fase, con la duración media en segundos impresa dentro del bloque y el color según su clase Roofline. La franja inferior marca el dispositivo de esa fase, y en las aplicaciones conjuntas distingue el tramo que corre en CPU del que corre en GPU. La fracción de tiempo en fases *memory_bound* varía entre 15 % en la aplicación B de CPU y 71 % en la A, lo que acota el ahorro posible de una política que actúa solo en esa clase.

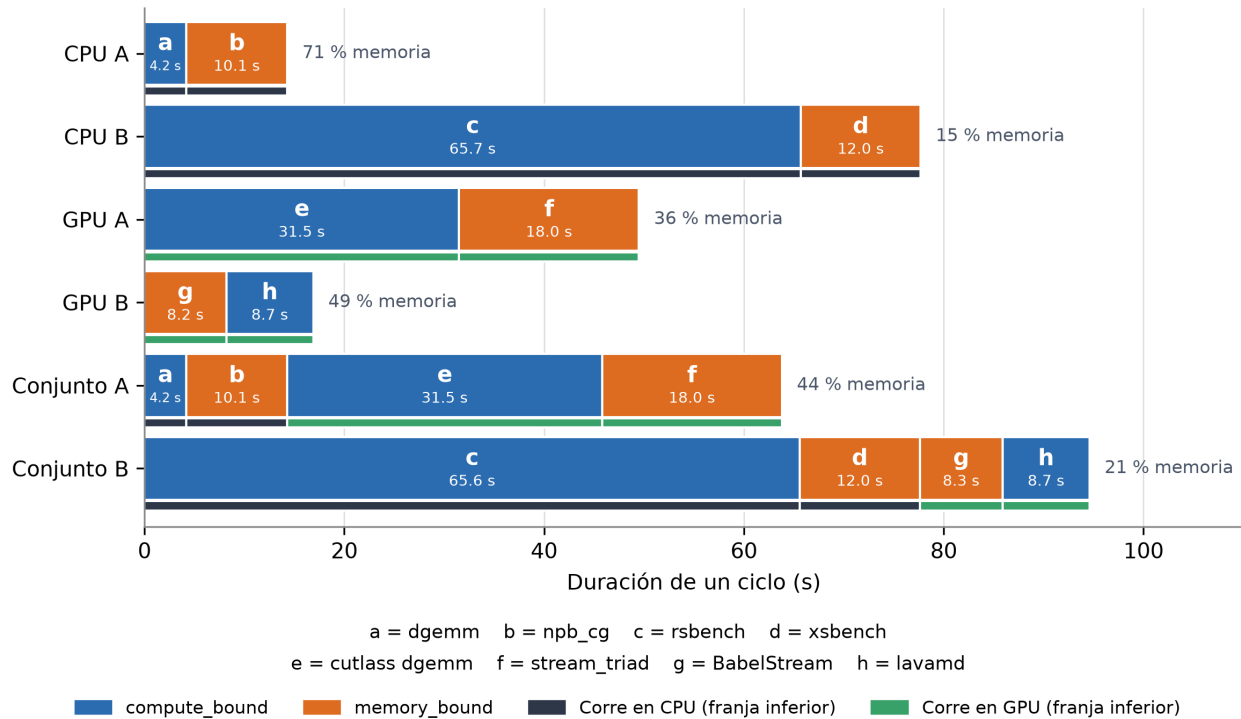


Figura 3.14: Un ciclo de cada aplicación, por fase (letra del kernel, clave debajo; color según su clase; franja inferior según el dispositivo). A la derecha, la fracción del ciclo en fases *memory_bound*. Fuente: elaboración propia.

3.4.2. Clasificación de fase con el agente en el lazo

Cada decisión del agente se puntúa contra la fase real correspondiente. En CPU, el agente acierta entre 96.7 y 97.5 % de las decisiones sobre las familias vistas y entre 99.6 y 99.8 % sobre las inéditas (dos kernels por aplicación, ponderadas por ventanas, y por eso no comparables con la exactitud LOFO). La diferencia favorable a las inéditas no debe leerse como mejor generalización. En la aplicación B, una sola fase de cómputo ocupa el 85 % del tiempo, y ponderar por ventanas la hace dominante. El resultado depende entonces de cómo se separan esos dos kernels, no de familias arbitrarias. En GPU no hubo ninguna decisión errónea entre las que el agente tomó, pero se abstuvo con frecuencia, con dos de doce fases en A y entre cuatro y cinco de doce en B (en el alcance conjunto, tres o cuatro de veinte en A y nueve de veinte en B). Toda fase, en cualquier alcance, recibió al menos una decisión. La exactitud del brazo activo y la del brazo sombra son indistinguibles, como corresponde a un clasificador cuya entrada no depende de la acción del agente.

3.4.3. Tiempo, energía, EDP y costo del agente frente a la base

La Figura 3.15 reporta la duración, las energías de CPU y de GPU por separado y las razones frente a la base de la matriz inicial, con el agente en su configuración final. Tres hechos separan los alcances:

- En **GPU** el agente activo no cambia el resultado. El EDP del nodo queda en 1.00 frente a la base en A y en B (y el brazo sombra, en 1.01 y 1.00). El agente de GPU no impone sobrecarga medible.
- En **CPU** el brazo activo queda en 1.15 (A) y 1.06 (B) veces el EDP de la base, con un aumento de 5.1 y 1.6 % en tiempo y de 11 y 5 % en energía de CPU.
- En el alcance **conjunto** el EDP queda en 1.04 (A) y 1.06 (B) veces el de la base. Con cinco repeticiones por brazo, la prueba de Mann–Whitney da $p = 0,0079$ para sombra, activo y activo sin piso frente a la base, en A y en B. Las cinco corridas de cada brazo quedaron por encima de las de la base.

El brazo sombra, que decide pero no escribe ninguna frecuencia, ya presenta la mayor parte del exceso de CPU (1.10 y 1.05 de EDP en A y B). El agente de CPU añade entre 0.4 y 1.4 % de tiempo y entre 5 y 9 % de energía de CPU (matriz inicial), un costo de mantenerlo que no depende de la frecuencia que se escriba, mientras que el de GPU no presentó costo medible (sombra entre 1.002 y 1.010 del EDP de la base). El agente de CPU usa por defecto un único hilo de inferencia. Con el conjunto de hilos por defecto de ONNX Runtime, cada hilo espera de forma activa entre inferencias y compite con la aplicación por los núcleos. La acción de frecuencia añade entre +0,8 % y +5,1 % (Figura 3.15; razón activo/sombra de 1.051 en CPU con kernels vistos, 1.008 en CPU con inéditos y 1.009 en el alcance conjunto, en ambos). Nunca mejora sobre solo observar, y el máximo es en CPU con kernels vistos, donde bajar el reloj en memoria empeora el EDP 5 %. Usar F0 también en *memory* no cambia el resultado de forma apreciable, y quitar el piso de coordinación lo empeora (Escenario C, tabla 2.8, abajo). Una segunda medición de la base (*base sin turbo*, el mismo estado) quedó entre 0.995 y 1.001 de la primera en EDP en las cinco combinaciones compuestas comparables del Escenario C y del Escenario E (se excluye la aplicación de familias inéditas del Escenario E, abreviada E-B más adelante, cuya mediana de tres repeticiones no es comparable por la bimodalidad de *myocyte*, sección 3.4.5). Eso acota la repetibilidad de la medición.

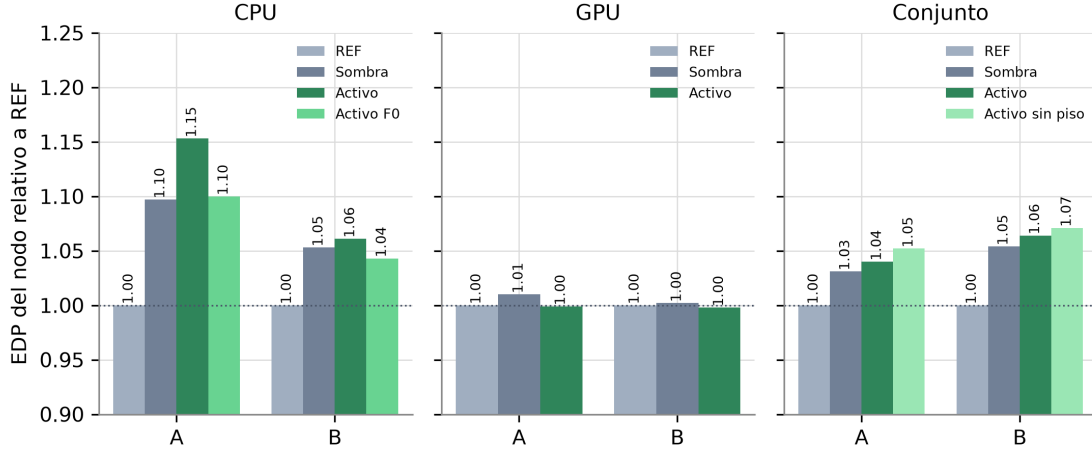


Figura 3.15: EDP del nodo, $(E_{\text{CPU}} + E_{\text{GPU}})T$, relativo a la base, por alcance, aplicación (A: vistos, B: inéditos) y brazo, en la matriz inicial. La línea punteada marca la paridad con la base.

Una repetición independiente de los alcances de CPU y conjunto (Escenario C, tabla 2.8), con el mismo agente y tres repeticiones por brazo, reprodujo estas razones con diferencias menores de 0.013 y con la misma exactitud de clasificación (97.7 % en A y 99.8 % en B en CPU, sin errores en las decisiones de GPU). En ella, bajar a F1 en memoria en CPU con kernels vistos dio 1.154 de EDP frente a 1.085 al mantener F0, y quitar el piso de coordinación en el alcance conjunto lo empeoró entre 0.5 y 1.6 puntos.

3.4.4. Potencial de la GPU

La ganancia máxima que la política de GPU puede aportar está acotada por la Fase 2. Fijar el reloj en F1 (1260 MHz) durante fases *memory_bound* mejora el EDP en 8.9 % (IC95 3.0 a 13.3 %), y los niveles más bajos no mejoran o empeoran (Figura 3.16, izquierda). El efecto sobre una aplicación completa se reduce por la fracción de su tiempo en memoria. Con 36 % (A) y 49 % (B) de fases *memory_bound*, la ganancia esperada de EDP de GPU es de 3.2 % y 4.3 %. La ganancia medida con el agente activo fue de 0.5 % en A y de 3.0 % en B en EDP de GPU, y de 0.1 % y 0.2 % en el EDP del nodo, que incluye la energía de CPU que el agente de GPU no modifica (Figura 3.16, derecha). Con tres repeticiones ninguna se separa de cero. En A, una de las tres repeticiones empeoró (1.5 %) y las otras dos mejoraron 1.4 %. En B las tres mejoraron (de 1.5 a 4.2 %), pero el brazo sombra, que no escribe frecuencia, ya queda 0.8 % por debajo de la base, del orden de la dispersión de la base entre repeticiones (1.3 % en energía de GPU).

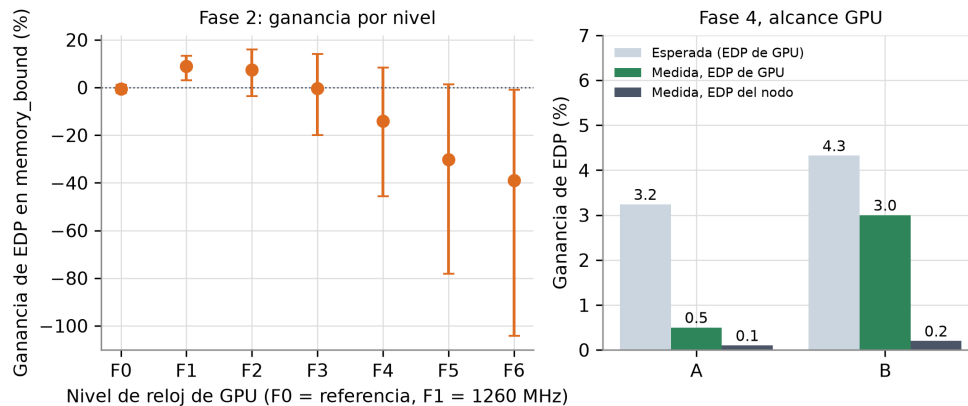


Figura 3.16: Izquierda: ganancia de EDP por nivel de reloj de GPU en fases *memory_bound* (Fase 2, IC95 sobre familias). Derecha: ganancia esperada de EDP de GPU (fracción de tiempo en memoria por la ganancia de F1) frente a la medida con el agente activo en el alcance GPU, en EDP de GPU y en EDP del nodo (medianas de tres repeticiones).

En A, el agente clasificó las fases de *triad* como memoria y aplicó F1 en cinco de las seis (en una repetición se abstuvo en ambas), y aun así la ganancia medida queda por debajo de la esperada. Con tres repeticiones no puede separarse de cero y su causa no está aislada. En B, las fases de BabelStream, de 8 s, se resolvieron con una sola decisión cada una, y en cuatro de las seis esa decisión fue una abstención, así que el reloj no se bajó. Con esa cobertura de decisión, de dos de seis fases, la ganancia esperada es del orden de 1.4%, no de 4.3%. La ganancia posible de la GPU depende entonces de dos condiciones, que la aplicación pase una parte apreciable del tiempo en fases largas *memory_bound* y que el agente no se abstenga en ellas. El escenario E de la sección 3.4.5 evalúa ambas.

3.4.5. Escenario E: GPU dominada por fases de memoria

El Escenario E (tabla 2.8) evalúa la política de GPU en aplicaciones dominadas por fases *memory_bound* largas, que es donde bajar el reloj tiene más tiempo para rendir. Sus dos aplicaciones se abrevian E-A, de familias vistas, y E-B, de familias inéditas; ambas siglas se usan así en el resto del capítulo y en la discusión. La primera decisión de GPU llegó entre 13 y 16 s después del inicio de cada fase de memoria (la ventana de observación sostenida es de 3 s). Por eso las fases de 46 a 60 s pudieron pasar entre 65 y 78 % de su duración en F1.

La Tabla 3.5 resume las dos aplicaciones declaradas antes de medir. En B, *myocyte* recibe una etiqueta de memoria por kernel pero casi no ocupa la GPU, y solo el 42 % del ciclo corresponde a memoria con actividad GPU. Esta diferencia entre etiqueta y oportunidad de actuación condiciona la interpretación del resultado.

Tabla 3.5: Composición del escenario E. Duraciones aproximadas por fase medidas antes de la matriz.

Aplicación	Fases de memoria	Fase de cómputo	Ciclo en memoria
A (vistas)	Stencil (58–60 s), SpMV (46 s), AXPY (47–48 s)	DGEMM (31 s)	84 %
B (inéditas)	BabelStream (36 s), Myocyte (40–41 s)	LavaMD (9 s)	89 % por etiqueta; 42 % con actividad de GPU

Familias vistas (E-A). Un confirmatorio declarado antes de medir, con cinco bloques aleatorizados y cinco repeticiones válidas por brazo, dio para el agente activo 0.9257 de energía de GPU, 0.9883 de duración y 0.9558 de EDP del nodo frente a la base, una reducción de 4.42 %. El brazo sombra quedó en 0.9970, así que el efecto no se debe al costo de observar. Las cinco diferencias activo–base fueron favorables, y con bloques como unidad la prueba de signos da $p = 0,0625$, el menor valor posible con cinco bloques. La media geométrica de las cinco razones de EDP del nodo es 0.962 (IC95 de 0.946 a 0.977; $p = 0,002$ en la prueba t pareada sobre el logaritmo), una reducción de 2.3 a 5.4 %, y la de energía de GPU es 0.928 (IC95 de 0.923 a 0.932). El intervalo del brazo sombra incluye la paridad (0.980 a 1.011). Una medición exploratoria previa de tres repeticiones señaló en el mismo sentido (0.935 de energía de GPU y 0.990 de EDP del nodo; Figura 3.17).

El ahorro del nodo es modesto porque la CPU consume el 64 % de la energía y no se modifica. En la medición exploratoria, con el agente de CPU en observación, su costo (3.4 % más de energía de CPU y 0.4 % más de tiempo) iguala el ahorro de GPU, y el EDP del nodo vuelve a 1.003. Conviene, por tanto, un agente de GPU sin agente de CPU, coherente con la política de no actuar sobre la CPU.

El brazo F1 fijo, previsto en el protocolo, no produjo celdas válidas, porque el reloj observado bajo carga quedó fuera del intervalo de 1230–1290 MHz exigido en cerca del 21 % de las muestras de cada bloque. Por eso el confirmatorio reproduce la mejora frente a la base, pero no permite establecer la ventaja de cambiar por fase frente a mantener F1 toda la aplicación. La premedición diagnóstica de F1 fijo anticipó el problema (en DGEMM el reloj observado con F1 fue indistinguible de la base) y no se usa para estimar el efecto del agente.

Familias inéditas (E-B). La fase de *myocyte* dura unos 41 s o unos 50 s sin relación con el brazo, con valores intermedios de 42 a 44.5 s, así que una mediana de tres repeticiones no

permite comparar, y razones como 0.84 de EDP solo reflejan qué repeticiones lentas cayeron en cada brazo. Por eso se agruparon las repeticiones de las tres corridas de esta aplicación con la misma configuración (la etapa E, una repetición de ocho bloques y una corrida de diagnóstico), con 21 repeticiones de la base y de sombra y 11 del brazo activo (Figura 3.17). Sombra y activo quedan en 1.028 y 1.016 del EDP de la base. En duración, la razón frente a esa misma base es de 1.017 para sombra y de 1.012 para activo, sin diferencia distinguible ($p = 0,21$ y $0,55$ en EDP, Mann–Whitney). En energía de GPU, la razón es de 1.006 para sombra y de 1.000 para activo. La fracción de repeticiones lentas es parecida en los tres brazos (5 de 21 en la base, 4 de 21 en sombra y 3 de 11 en el activo). La corrida de diagnóstico, con telemetría de GPU, de la frecuencia, del núcleo donde corre la aplicación y de los demás hilos del nodo, no encontró diferencia entre repeticiones lentas y rápidas. Fijar el proceso del agente de GPU a un núcleo fuera de los núcleos 0 a 5 tampoco las elimina de forma distinguible. La variabilidad es entonces de la aplicación, y su causa no está identificada. BabelStream recibió una decisión de memoria en 2 de las 11 repeticiones del brazo activo (confianza 1.00), con 11 a 13 % menos de energía de GPU que una repetición rápida de la base, y en las otras nueve se abstuvo (confianza 0.56 a 0.67). *Myocyte* no superó el umbral de actividad GPU y no recibió decisión. El límite observado es la cobertura de decisión sobre familias inéditas, no un empeoramiento por F1.

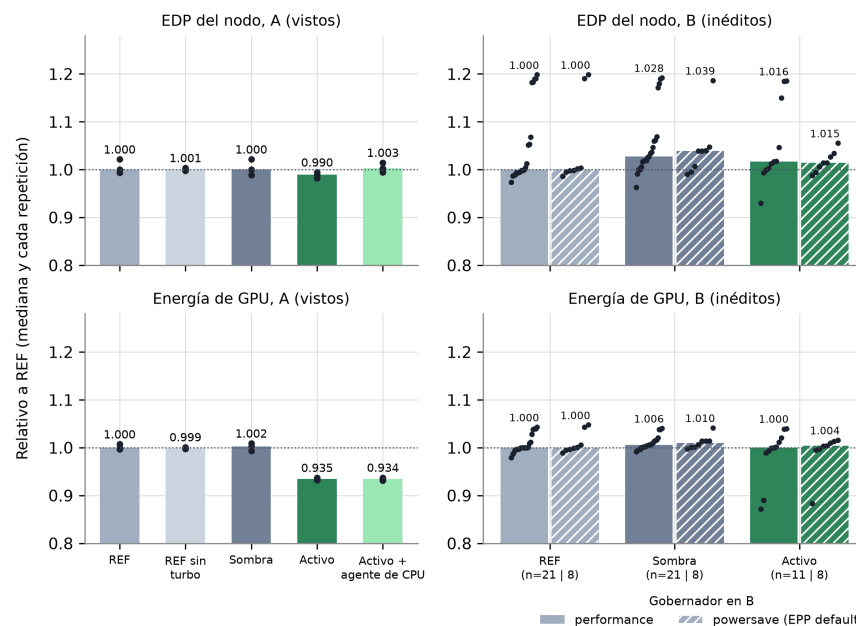


Figura 3.17: Escenario E. EDP del nodo (arriba) y energía de GPU (abajo) relativos a REF, con la mediana (barra) y cada repetición (punto), para familias vistas (A) e inéditas (B). En B, cada brazo muestra las repeticiones bajo performance (barras lisas) junto a las ocho bajo powersave con EPP default (barras rayadas), ambas frente a la REF de su propio gobernador.

Contraste con el gobernador nativo powersave (E-B). La comparación anterior usa *performance* como gobernador nativo en los núcleos delegados. Para contrastarla con otro gobernador nativo de Linux bajo *intel_pstate*, se repitió la matriz de E-B, con los mismos tres kernels inéditos, los mismos cinco brazos del escenario E y ocho bloques aleatorizados, con los 12 núcleos delegados en *powersave* y el perfil de energía-rendimiento (EPP) en *default*, que la plataforma resolvió a *balance_performance*. El techo de 3.2 GHz de los núcleos delegados se mantuvo igual que bajo *performance*. La columna B de la Figura 3.17 muestra cada brazo bajo *powersave* (barras rayadas, ocho repeticiones) al lado de las repeticiones agrupadas bajo *performance*, cada uno frente a la REF de su propio gobernador.

La REF bajo *powersave* queda en 1.004 de la duración y 1.002 del EDP frente a la REF bajo *performance*, sin diferencia distinguible ($p = 0,32$ en duración y $p = 0,58$ en EDP, Mann-Whitney), y su energía de CPU baja levemente, a 0.993 ($p = 0,94$). La REF sin turbo bajo *powersave* queda en 1.005 del EDP de la REF del mismo gobernador, así que con el techo de 3.2 GHz el turbo tampoco cambia el resultado. El gobernador nativo por sí solo no cambia la base en este escenario, lo que es consistente con que la CPU pasa la mayor parte del ciclo esperando a la GPU en fases dominadas por memoria. La bimodalidad de *myocyte* también aparece bajo *powersave*, con 2 de 8 repeticiones lentas en la REF, 1 de 8 en sombra y ninguna en el activo, de modo que no depende del gobernador.

Dentro de *powersave*, sombra y activo quedan en 1.039 y 1.015 del EDP de su propia REF, y en 1.020 y 1.011 de su duración. Ninguna de las cuatro razones se distingue de la REF ($p = 0,57$ en ambos casos, Mann-Whitney), y las dos de EDP quedan muy cerca de los 1.028 y 1.016 medidos bajo *performance*. El patrón del brazo activo se repite bajo los dos gobernadores. Su energía de GPU queda en 1.004 bajo *powersave* y en 1.000 bajo *performance*, con una sola repetición de *powersave* 12 % por debajo de la REF, del mismo orden que las dos repeticiones de *performance* en que BabelStream recibió decisión de memoria, y en ninguno de los dos casos compensa el costo de observación del brazo sombra. El activo con el agente de CPU en observación quedó en 1.176 del EDP, pero 5 de sus 8 repeticiones cayeron en la fase lenta de *myocyte*. Con solo las repeticiones rápidas, queda en 1.002 del EDP de las repeticiones rápidas de la REF, así que ese valor refleja la bimodalidad y no un costo del agente ($p = 0,10$ con todas las repeticiones).

3.4.6. Escenario D: LAMMPS como aplicación de terceros

El Escenario D (tabla 2.8) evalúa LAMMPS 2023 con el paquete GPU del clúster, sin imponer fronteras de fase al programa. Se ejecutaron tres bloques de las entradas *chain*, *eam*, *lj* y *rhodo*, con la base, sombra, agente de GPU activo y una variante que mantiene el

agente de CPU en observación. Todas las 48 celdas terminaron con código de aplicación y de daemon válido, y restauraron el estado de frecuencia. La Figura 3.18 muestra que el brazo activo de GPU queda entre 1.003 y 1.009 de la base en EDP, según la entrada. No se observa una mejora persistente, y la variante con el agente de CPU en observación incrementa el EDP entre 2.9 y 3.4 %. En *chain* y *rhodo* el agente no emitió decisiones, y en *lj* y *eam* solo emitió *compute_bound*, así que no aplicó F1. Este resultado extiende la evaluación a una aplicación externa, pero no mide el efecto de la política GPU sobre una fase actuada.

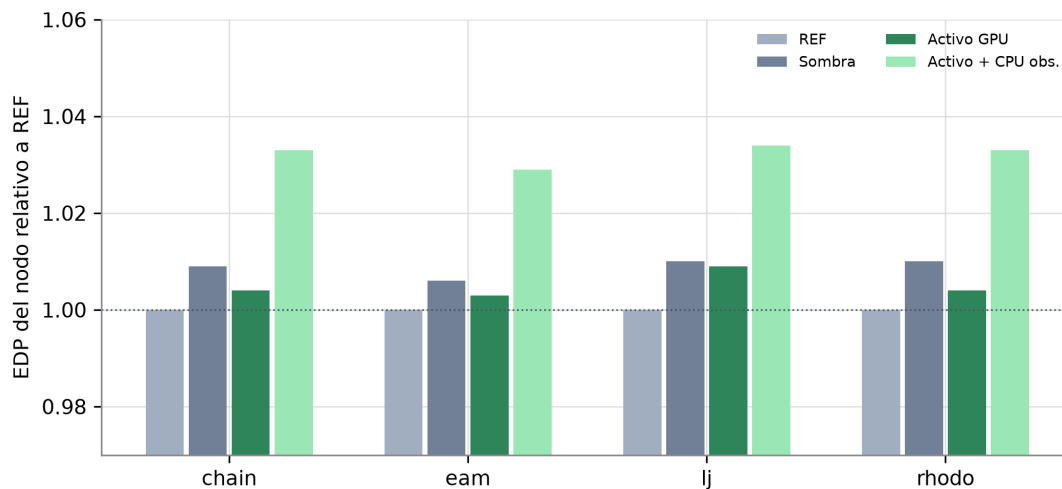


Figura 3.18: EDP mediano de LAMMPS por entrada y brazo, relativo a la base. Fuente: elaboración propia.

3.4.7. CloverLeaf CUDA Fortran como aplicación continua

CloverLeaf CUDA Fortran se evaluó como una aplicación continua de dinámica de fluidos, sin fronteras de fase impuestas al programa. El confirmatorio ejecutó cinco bloques aleatorizados con la base, sombra y activo. Las quince corridas terminaron con código de aplicación válido, restauración correcta del reloj y solución numérica reproducible. La energía cinética del fluido simulado, que CloverLeaf informa al terminar, coincidió en las quince corridas dentro de una tolerancia relativa de 10^{-8} . En cada corrida activa el agente emitió una decisión *memory_bound* con confianza 1.00 tras aproximadamente 3 s y solicitó 1260 MHz. Tras esa decisión, las 433 o 434 muestras NVML bajo carga de cada bloque permanecieron en 1260 MHz. CloverLeaf no aporta una etiqueta Roofline independiente por fase. Por eso este experimento verifica la actuación y su efecto energético, pero no puntúa la exactitud de esa decisión.

El brazo activo mantuvo la duración en 1.0015 de la base, redujo la energía de GPU a

0.9393 y dejó el EDP del nodo en 0.9625, una reducción de 3.75 %. El brazo sombra quedó en 0.9958 de EDP (media geométrica por bloque de 0.997, IC95 de 0.995 a 0.999), dentro del rango de repetibilidad de la base (sección 3.4.3), así que no se observa un costo apreciable de mantener el agente sin actuar. Las cinco diferencias de EDP activo–base fueron favorables, y la prueba de signos por bloques da $p = 0,0625$, el menor valor posible con cinco bloques. La media geométrica de las razones de EDP del brazo activo es 0.961 (IC95 de 0.954 a 0.968; $p < 0,001$ en la prueba t pareada sobre el logaritmo), una reducción de 3.2 a 4.6 %, y la de energía de GPU es 0.936 (IC95 de 0.924 a 0.947). La Figura 3.19 separa el resultado entre CPU y GPU. La actuación reduce la energía del dispositivo sin modificar de forma apreciable el tiempo de ejecución.

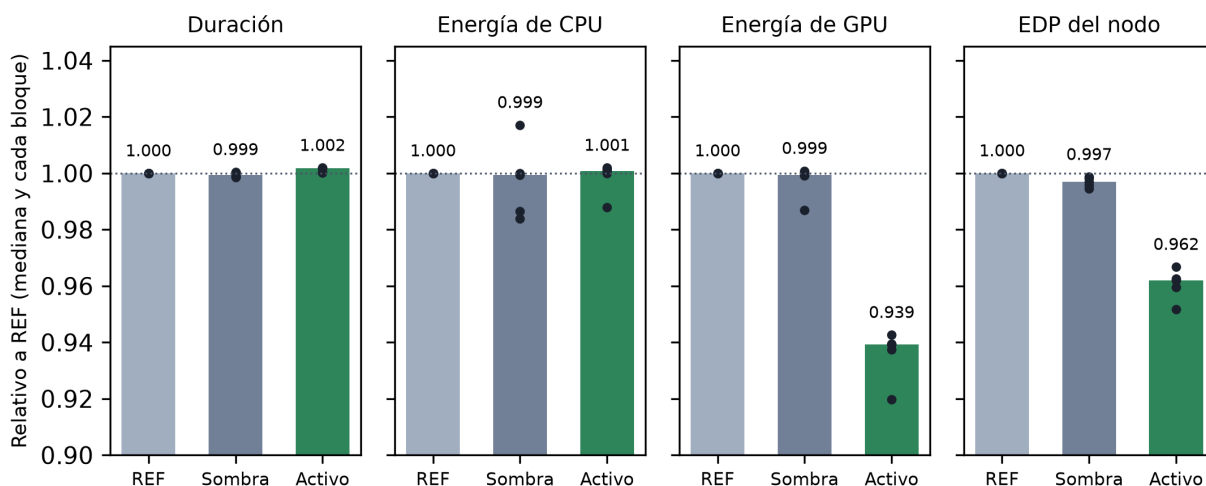


Figura 3.19: CloverLeaf CUDA Fortran: duración, energía de CPU, energía de GPU y EDP del nodo de cada brazo relativos a la base. La barra es la mediana de los cinco bloques, los puntos son los bloques individuales y la línea punteada marca la paridad con la base. Fuente: elaboración propia.

Capítulo 4

Discusión

4.1. Validez de la medición

Los enfoques de selección de frecuencia revisados [5, 10, 32] centran su evaluación en la decisión. En este trabajo también se comprobó su premisa física, que es que el hardware ejecute la frecuencia solicitada. Restringir solo los procesadores lógicos delegados de CPU no alteró el rendimiento, y fue necesario restringir también sus hermanos de *SMT*. En GPU, el candado sostuvo los relojes solicitados durante la prueba de actuación, aunque el brazo F1 fijo del confirmatorio posterior falló la verificación bajo carga. Estas observaciones justifican aceptar una corrida por el reloj medido, y no por el comando de actuación. La verificación independiente de los techos Roofline y del presupuesto de contadores cumple la misma función de protección de la etiqueta.

El catálogo GPU mostró, además, que el tipo de dato declarado por un kernel no siempre describe la precisión que ejecuta el hardware (sección 3.2.1). Para un kernel cercano al punto de inflexión, como *rodinia_hotspot*, esa diferencia puede cambiar la etiqueta Roofline. La caracterización de precisión efectiva debe preceder al entrenamiento cuando la etiqueta depende del número de operaciones por precisión.

4.2. Generalización de los clasificadores

El resultado de CPU cambia con la unidad de separación. Mezclar intervalos al azar da una exactitud mucho mayor que dejar fuera una familia (sección 3.1.4). Los 1.31 millones de intervalos proceden de 30 familias, y el último protocolo mide mejor la llegada de un algoritmo nuevo. Cambiar el modelo, ampliar las variables o buscar hiperparámetros no produjo una

mejora distinguible bajo ese protocolo. La diversidad de familias sí elevó el resultado. Es consistente con que la etiqueta se defina por FLOPs sobre bytes de DRAM, mientras que el modelo observa fallos de caché, ciclos detenidos e instrucciones retiradas, que no guardan una relación fija con esa razón cuando cambia el algoritmo. La abstención reduce algunos errores, pero no corrige los de alta confianza en familias inéditas.

En GPU, la selección cambió al considerar el lazo de control. La regresión logística de cinco señales tenía buena exactitud y calibración estática, pero leía reloj y potencia, dos consecuencias de la acción que debía decidir. El bosque desplegado usa tres entradas invariantes y evita esa retroalimentación. Su umbral de 0.90 sacrifica cobertura para sostener la precisión de las decisiones emitidas. La latencia de inferencia es pequeña frente a los segundos que consume detectar una fase y actuar, así que mejorar solo el tiempo de predicción tendría poco efecto sobre fases breves. La abstención, que el agente no reevalúa en la misma fase, sigue siendo la limitación principal ante memoria inédita.

4.3. Alcance de la ganancia energética

La diferencia entre CPU y GPU procede de la respuesta física de cada dispositivo. En CPU, el piso estático de potencia domina. Reducir el reloj deja poco margen energético y prolonga la ejecución, así que la política por clase es *no actuar*. En GPU, el piso pesa menos, queda más potencia dinámica por recortar, y las fases limitadas por memoria responden poco al reloj SM. Por eso F1 mejoró su EDP agregado en la campaña de kernels (sección 3.2.4). La etiqueta Roofline identifica una oportunidad, pero no garantiza que todos los kernels de la clase respondan igual, y `rodinia_gaussian` perdió en la validación por familia retirada.

La Fase 4 delimitó dónde se traduce esa oportunidad en una aplicación. El brazo sombra separó el costo de observar del de actuar: el agente GPU no mostró costo medible, y el de CPU añadió entre 5 y 9% de energía de CPU en la matriz inicial. E-A (familias vistas del Escenario E, sección 3.4.5) y CloverLeaf (sección 3.4.7) redujeron el EDP del nodo frente a la base, y el ahorro del dispositivo se diluyó porque la energía de CPU apenas cambió; en ambos casos los cinco bloques favorecieron al agente y el IC95 de la reducción, calculado después de medir, excluye la paridad. El brazo F1 fijo de E-A no fue válido, así que no se pudo comparar la conmutación por fase con mantener ese nivel. LAMMPS y las familias inéditas quedaron cerca de paridad porque el agente no aplicó F1 de forma sostenida, y en estas últimas cambiar el gobernador nativo de *performance* a *powersave* no alteró la base ni el patrón del agente. La mejora observada corresponde a fases de memoria largas con decisiones confiables, no a cualquier aplicación GPU.

Capítulo 5

Conclusiones

El instrumento CPU–GPU quedó validado mediante comprobaciones de contadores, frecuencia efectiva y calibración Roofline antes de usar sus medidas para entrenar. La campaña CPU produjo 1470 corridas aceptadas y 1 309 755 intervalos elegibles; la campaña GPU reunió 483 corridas de 16 familias. Esta base permitió separar la calidad de las etiquetas, la generalización de los modelos y el efecto energético de la actuación, tres propiedades que no se deducen unas de otras.

En CPU, el clasificador XGBoost obtuvo 0.728 de exactitud balanceada por celda al dejar una familia algorítmica fuera, y 0.738 sobre el 72.2 % de los intervalos decididos con el umbral de confianza de 0.85. La diversidad de familias influyó más que cambiar el algoritmo o ajustar los hiperparámetros. La política resultó *no actuar* en las dos clases Roofline, porque unos 85 W de potencia de paquete, el 84 % de la potencia al mayor nivel fijo, permanecen al reducir el reloj. Incluso elegir el mejor nivel para cada kernel ofrece solo 2.2 % de ganancia media. La Fase 4 confirmó que, en las aplicaciones compuestas medidas, actuar sobre la frecuencia de CPU no mejoró el EDP frente a la base.

En GPU, el modelo desplegado es un *random forest* de tres señales invariantes, con exactitud balanceada por celda de 0.819 sobre corridas completas y error de calibración esperado de 0.0868. La regresión logística de cinco señales fue una referencia estática competitiva (0.792), pero su reloj y su potencia cambian cuando actúa el agente. Con el umbral de 0.90, la exactitud sobre las decisiones aceptadas llegó a 0.900, con una cobertura media por celda de 65.7 %. La política fijó F1 para *memory_bound*, con una mejora de EDP de GPU de 8.9 % (IC95 de 3.0 a 13.3 %) en la campaña de kernels y de 6.5 % (IC95 de −4.0 a 13.1 %, que incluye el cero) al elegir el nivel sin la familia evaluada. Para *compute_bound* se conservó el *governor* nativo. El clasificador aproxima con lecturas NVML una etiqueta que de otro modo

exige perfilar con Nsight Compute, que en este trabajo alargó unas 60 veces la ejecución de un *kernel*, y el agente de GPU lo hace sin costo medible sobre la aplicación (brazo sombra entre 1.002 y 1.010 del EDP de la base). Esta diferencia entre dispositivos responde al menor peso del piso de potencia de la GPU y a la escasa sensibilidad al reloj de cómputo de las cargas limitadas por memoria.

El agente quedó implementado y se evaluó en los brazos base, sombra y activo. En el confirmatorio prospectivo de E-A redujo la energía de GPU 7.43 % y el EDP del nodo 4.42 % frente a la base. En CloverLeaf CUDA Fortran, una aplicación externa continua, redujo esas magnitudes 6.07 y 3.75 %. En ambos casos los cinco bloques favorecieron al agente. La prueba de signos declarada dio $p = 0,0625$, el menor valor posible con cinco bloques. La estimación del efecto por bloques, añadida después de medir, da reducciones del EDP del nodo con IC95 de 2.3 a 5.4 % en E-A y de 3.2 a 4.6 % en CloverLeaf, ambas distintas de cero ($p = 0,002$ y $p < 0,001$ en la prueba t pareada). E-A se diseñó en condiciones favorables identificadas de antemano, así que estas cifras no estiman el efecto sobre una aplicación cualquiera. En CloverLeaf se verificaron la actuación y la salida numérica, y no existe una etiqueta física independiente para puntuar su clasificación de fase. El brazo F1 fijo de E-A no sostuvo el reloj exigido bajo carga, así que no se pudo comparar el cambio por fase con un nivel fijo. LAMMPS y las aplicaciones con familias GPU inéditas quedaron cerca de la paridad, y en las inéditas la abstención limitó el ahorro. La mejora observada depende de fases de memoria suficientemente largas y de decisiones de alta confianza.

5.1. Limitaciones

Los intervalos CPU proceden de 30 familias y no son unidades independientes de generalización. Su etiqueta tiene la granularidad de *uncore*, cercana a 10 ms, y la abstención no elimina errores de alta confianza sobre familias nuevas. El modelo GPU clasifica corridas completas, no intervalos, y se evaluó solo con validación por familia retirada sobre 16 familias; NVML no observa los contadores que definen directamente el régimen Roofline, y algunos kernels cambian de régimen al variar la frecuencia. La política F1 se validó con ocho familias *memory_bound*, cuatro de ellas núcleos de elaboración propia. Los dos experimentos favorables de aplicación tuvieron cinco bloques cada uno, con lo que la prueba de signos no podía bajar de $p = 0,0625$ y el intervalo del efecto se calculó después de medir, sin poder verificar la normalidad, y el contraste contra F1 fijo quedó inválido. La base se midió con los dos gobernadores que expone *intel_pstate* en esta plataforma, *performance* y *powersave*, ambos con turbo desactivado; *ondemand* y *schedutil* no están disponibles con ese controlador. En E-B, el único escenario con resultado bajo ambos gobernadores, *powersave* no mostró diferencia

frente a *performance*. Todas las mediciones proceden de un único nodo, y las aplicaciones de familias vistas reutilizan kernels del entrenamiento. Estas condiciones acotan la extrapolación a otras aplicaciones y plataformas.

5.2. Trabajo futuro

La siguiente validación debería ampliar las familias GPU externas, en especial las fases *memory_bound* que hoy provocan abstención, y evaluar una sola vez el modelo y el umbral congelados. También conviene repetir el contraste entre la base, el agente y F1 fijo con verificación continua del reloj bajo carga. Para mejorar la respuesta en fases breves se puede medir por separado la latencia de detección, de recolección de señales y de actuación, y estudiar una reevaluación controlada de las abstenciones. CUPTI Activity podría aportar fronteras de fase para construir etiquetas independientes, previa medición de su intrusión, mientras NVML se mantiene como entrada ligera del agente. En CPU, la salida *revisar* tendría contenido si el agente midiera la etiqueta física de una fracción mínima de intervalos. Además, conviene ampliar el catálogo con familias elegidas por su distancia a los perfiles cubiertos, reservar algunas para una prueba externa sellada (las seis familias de cómputo denso de RAJAPerf provienen de una misma suite) y medir la latencia de inferencia de GPU dentro del agente y no en el intérprete de Python. Cualquier revisión de la política requiere reducir el costo de observación y demostrar un margen de ahorro superior al piso de potencia medido.

Bibliografía

- [1] O. S. Simsek, J.-G. Piccinali, and F. M. Ciorba, “Increasing energy efficiency of astrophysics simulations through gpu frequency scaling,” in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, pp. 1826–1834.
- [2] M. T. Costa *et al.*, “Characterizing the impact of gpu power management on an exascale system,” in *Proc. SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops ’25)*, New York, NY, USA, 2025, pp. 1524–1533.
- [3] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama, “Mcbound: An online framework to characterize and classify memory/compute-bound hpc jobs,” in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, pp. 1–15.
- [4] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccion, “Evaluation of dvfs techniques on modern hpc processors and accelerators for energy-aware applications,” *Concurrency and Computation: Practice and Experience*, vol. 29, no. 12, p. e4143, 2017.
- [5] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen, “An automated and portable method for selecting an optimal gpu frequency,” *Future Generation Computer Systems*, vol. 149, pp. 71–88, 2023.
- [6] R. Gonzalez, B. M. Gordon, and M. A. Horowitz, “Supply and threshold voltage scaling for low power CMOS,” *IEEE Journal of Solid-State Circuits*, vol. 32, no. 8, pp. 1210–1216, 1997.
- [7] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

- [8] K. Goto and R. A. van de Geijn, “Anatomy of high-performance matrix multiplication,” *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 1–25, 2008, article 12.
- [9] O. Antepara, L. Oliker, and S. Williams, “Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200,” in *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1205–1216.
- [10] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás, “DVFS-aware application classification to improve GPGPUs energy efficiency,” *Parallel Computing*, vol. 83, pp. 93–117, 2019.
- [11] National Energy Research Scientific Computing Center (NERSC), “Roofline performance model,” NERSC Documentation, 2026, accessed: Aug. 7, 2026. [Online]. Available: <https://docs.nersc.gov/tools/performance/roofline/>
- [12] V. M. Weaver and S. A. McKee, “Can hardware performance counters be trusted?” in *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, Seattle, WA, USA, 2008, pp. 141–150.
- [13] V. M. Weaver, “Self-monitoring overhead of the linux perf event performance counter interface,” in *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2015, pp. 102–111.
- [14] Intel Corporation, “Intel 64 and ia-32 architectures software developer’s manual, volume 3b: System programming guide, part 2 (performance monitoring, nmi watchdog interaction with fixed-function and general-purpose counters),” 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [15] T.-Y. Liu, J. Guo, and B. Huang, “Efficient cross-platform multiplexing of hardware performance counters via adaptive grouping,” *ACM Transactions on Architecture and Code Optimization*, vol. 21, no. 1, pp. 1–26, 2024.
- [16] Linux man-pages project, “perf_event_open(2) — linux manual page,” 2026, accessed: Aug. 29, 2026. [Online]. Available: https://man7.org/linux/man-pages/man2/perf_event_open.2.html
- [17] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA, USA: No Starch Press, 2010.

- [18] S. McCarty, “Architecting containers part 1: Why understanding user space vs. kernel space matters,” Red Hat Blog, 2015, accessed: Apr. 3, 2025. [Online]. Available: <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters>
- [19] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le, “RAPL: Memory power estimation and capping,” in *Proc. 16th ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, 2010, pp. 189–194.
- [20] P. Thamm and U. Leser, “Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters,” *arXiv preprint arXiv:2505.09375*, 2025.
- [21] R. Hebbar and A. Milenković, “PMU-events-driven DVFS techniques for improving energy efficiency of modern processors,” *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 7, no. 1, pp. 1–31, 2022.
- [22] Linux Kernel Documentation, “CPU Performance Scaling,” 2026, accessed: Sep. 5, 2026. [Online]. Available: <https://docs.kernel.org/admin-guide/pm/cpufreq.html>
- [23] D. Velicka, O. Vysocky, and L. Riha, “Methodology for GPU frequency switching latency measurement,” in *Proc. 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2025, pp. 830–839.
- [24] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2020.
- [25] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: with Applications in R*, 2nd ed., ser. Springer Texts in Statistics. Springer, 2021.
- [26] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [27] P. Geurts, D. Ernst, and L. Wehenkel, “Extremely randomized trees,” *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.
- [28] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [29] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.

- [30] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems*, vol. 24, 2011, pp. 2546–2554.
- [31] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan, “Energy delay product,” in *Energy-Efficient High Performance Computing: Measurement and Tuning*, ser. SpringerBriefs in Computer Science. London, UK: Springer, 2013, ch. 6, pp. 51–55.
- [32] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang, “DRLCap: Runtime GPU frequency capping with deep reinforcement learning,” *IEEE Transactions on Sustainable Computing*, vol. 9, no. 5, pp. 712–726, 2024.
- [33] Linux perf project, “perf-stat(1) measurement overhead documentation,” 2026, accessed: Sep. 20, 2026. Man pages explicitly warn: “The overhead percentage could be high in some cases, for instance with small, sub 100ms intervals. Use with caution.” Intervals below 10ms incur syscall overhead that dominates measurement time in typical deployments. [Online]. Available: <https://man7.org/linux/man-pages/man1/perf-stat.1.html>
- [34] Intel Corporation, *3rd Gen Intel Xeon Processor Scalable Family, Codename Ice Lake, Uncore Performance Monitoring Reference Manual*, May 2021, revision 1.0. [Online]. Available: <https://cdrdv2-public.intel.com/679093/639778%20ICX%20UPG%20v1.pdf>
- [35] —, “perfmon: Performance monitoring events for intel architecture processors (event to fp_arith_inst_retired, familia ice lake),” 2026, accessed: Aug. 10, 2026. [Online]. Available: <https://github.com/intel/perfmon>
- [36] NVIDIA, “Nsight compute,” NVIDIA Developer Documentation, 2026, accessed: Aug. 8, 2026. [Online]. Available: <https://docs.nvidia.com/nsight-compute/>
- [37] D. C. Montgomery, *Design and Analysis of Experiments*, 9th ed. Wiley, 2017.
- [38] D. R. Roberts, V. Bahn, S. Ciuti, M. S. Boyce, J. Elith, G. Guillera-Arroita, S. Hauens-tein, J. J. Lahoz-Monfort, B. Schröder, W. Thuiller, D. I. Warton, B. A. Wintle, F. Hartig, and C. F. Dormann, “Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure,” *Ecography*, vol. 40, no. 8, pp. 913–929, 2017.
- [39] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

- [40] K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann, “The balanced accuracy and its posterior distribution,” in *2010 20th International Conference on Pattern Recognition*, 2010, pp. 3121–3124.
- [41] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.
- [42] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [43] M. Hollander, D. A. Wolfe, and E. Chicken, *Nonparametric Statistical Methods*, 3rd ed. John Wiley & Sons, 2014.
- [44] S. Plimpton, “Fast parallel algorithms for short-range molecular dynamics,” *Journal of Computational Physics*, vol. 117, no. 1, pp. 1–19, 1995.
- [45] A. C. Mallinson, D. A. Beckingsale, W. P. Gaudin, J. A. Herdman, J. M. Levesque, and S. A. Jarvis, “CloverLeaf: Preparing hydrodynamics codes for exascale,” in *Proceedings of the Cray User Group 2013*, Napa, CA, USA, May 2013. [Online]. Available: https://cug.org/proceedings/cug2013_proceedings/includes/files/pap130.pdf
- [46] Intel Corporation, “Roofline analysis with intel advisor,” Intel Advisor User Guide, 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/advisor/user-guide/2026-0/roofline-perspective-in-intel-advisor.html>
- [47] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.
- [48] D. Chicco and G. Jurman, “The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation,” *BMC Genomics*, vol. 21, no. 1, p. 6, 2020.